

UAX

**TRAB
AJO
FIN
DE
MÁST
ER**

**MUIA
24-25**

UNIVERSIDAD ALFONSO X EL SABIO

Business Tech

**Máster Universitario en
Inteligencia Artificial**



TRABAJO DE FIN DE MÁSTER

**Sistema de agentes inteligentes
basado en IA generativa para la
predicción de incendios forestales**

Ángel Bermúdez Caballero

**Director académico: Ángel Albarracín
Director de la empresa: Ricardo Picatoste (XRF)**

Noviembre de 2025

RESUMEN

Los incendios forestales representan hoy en día, una de las amenazas medioambientales más críticas, generando graves impactos tanto en el sector ecológico como en el sector económico y social. Este Trabajo Fin de Máster presenta el desarrollo de un sistema multiagente basado en inteligencia artificial generativa orientado y enfocado a la predicción del riesgo de incendios forestales. El proyecto se ha hecho en el periodo de prácticas curriculares del máster, en la empresa XRF.ai. Empresa que está especializada en la aplicación de tecnologías de IA tanto al ámbito de emergencias como en el de defensa, enfocado en la automatización y creación de entornos para el análisis de información estratégica ayudando de esta manera, a la toma de decisiones en entornos operativos y reales.

El sistema diseñado, denominado Agente de Emergencias, combina la capacidad de razonamiento autónomo y la generación de lenguaje natural a través del agente ReAct (Reason + Act), proveniente del framework LangChain y cuya fuente de recursos ha sido Azure OpenAI. Además, contiene diferentes herramientas como un módulo de geocodificación, análisis meteorológico, cálculo de riesgos y probabilidades y visualización geoespacial en una interfaz desarrollada con Streamlit. Gracias a coordenadas concretas obtenidas con la geocodificación y a partir de diversos datos meteorológicos obtenidos mediante la API Open-Meteo, el agente es capaz de estimar la probabilidad de ignición, evaluar el peligro de propagación y generar mapas de calor interpolados que reflejan la distribución espacial tanto del riesgo de incendio como de la probabilidad de ignición como cualquier dato meteorológico que se estime oportuno y todo en tiempo real.

Los resultados obtenidos nos permiten apreciar la capacidad de la IA generativa para operar como base en sistemas de predicción ambiental, mejorando precisión y la asistencia a la decisión en situaciones críticas. Este trabajo refleja el potencial más que evidente de tecnologías avanzadas de IA en los ámbitos de emergencias y defensa, demostrando su eficiencia para

fortalecer y llevar a otro nivel la resiliencia frente a amenazas naturales y contribuir a la seguridad operativa en este tipo de situaciones.

Palabras clave

IA generativa, sistema multiagente, LangChain, gestión de emergencias medioambientales, riesgo de incendios forestales.

ABSTRACT

Forest fires are currently one of the most critical environmental threats, causing serious impacts on the ecological, economic and social sectors. This Master's Thesis presents the development of an intelligent agent system based on generative artificial intelligence aimed at predicting the risk of forest fires. The project was carried out during the master's degree internship period at XRF.ai, a company specialising in the application of AI technologies in both the emergency and defence fields, focusing on the automation and creation of environments for strategic information analysis, thus assisting decision-making in operational and real-world environments.

The system designed, called `Agente_Emergencias`, combines autonomous reasoning and natural language generation through the ReAct (Reason + Act) agent, from the LangChain framework, whose resource source has been Azure OpenAI. It also contains various tools such as a geocoding module, meteorological analysis, risk and probability calculation, and geospatial visualisation in an interface developed with Streamlit. Thanks to specific coordinates obtained through geocoding and various meteorological data obtained through the Open-Meteo API, the agent is able to estimate the probability of ignition, assess the danger of spread, and generate interpolated heat maps that reflect the spatial distribution of both the risk of fire and the probability of ignition, as well as any meteorological data deemed appropriate, all in real time.

The results obtained allow us to appreciate the capacity of generative AI to operate as a basis for environmental prediction systems, improving accuracy

and decision support in critical situations. This work reflects the clear potential of advanced AI technologies in the fields of emergencies and defence, demonstrating their efficiency in strengthening and taking resilience to natural threats to another level and contributing to operational safety in such situations.

Keywords

Generative AI, Multi-Agent System, LangChain, Environmental Emergencies, Wildfire Risk

ÍNDICE

Capítulo 1: Introducción al Trabajo Fin de Máster	7
1. Justificación del proyecto realizado.....	7
2. Presentación del proyecto y sus contenidos	9
Capítulo 2: Objetivos del TFM	14
1. Objetivo general	15
2. Objetivos específicos	15
Capítulo 3: Marco Teórico	19
1. Agente orquestador	20
2. API Meteorológica para la obtención de datos en tiempo real	26
3. Procesos de geocodificación y obtención de coordenadas	28
4. Generación de mallas geográficas y muestreos espaciales	31
5. Cálculo de probabilidad de ignición y riesgo de incendio	34
6. Generación de mapas de calor e interpolación espacial	36
7. Visualización en la interfaz de Streamlit	38

Capítulo 4: Marco Metodológico	42
1. Población y muestra	42
2. Variables intervinientes e instrumentos de evaluación	44
3. Diseño experimental	45
4. Modelo de análisis de datos	46
Capítulo 5: Presentación de análisis, resultados y discusión	50
1. Análisis descriptivo de la muestra	50
2. Resultados obtenidos	52
3. Discusión y análisis de los resultados	58
Capítulo 6: Conclusiones	59
1. Limitaciones	61
2. Futuras líneas de mejora	63
3. Consideraciones finales	65
Bibliografía	67

Capítulo 1: Introducción al Trabajo Fin de Máster

El análisis y la gestión de riesgo de incendios forestales es un factor cada vez más prioritario en el área de ingeniería ambiental y en emergencias sobre todo en ciertas épocas del año. Más allá del impacto ecológico y mortífero que conllevan, los incendios son un fenómeno de alta complejidad sistémica que requiere y depende de gran cantidad de factores, influyen variables meteorológicas, topográficas y humanas con relación directa. En consecuencia, se exigen modelos capaces de ofrecer interpretaciones adaptativas antes de modelos puramente técnicos o descriptivos.

En los últimos años, la digitalización del acceso a información meteorológica ha provocado la posibilidad de implementaciones para el desarrollo de sistemas inteligentes orientado tanto al estudio como a la prevención y predicción de catástrofes naturales de este tipo. No obstante, al ser modelos de alta complejidad ocasionaban poca capacidad de adaptación y al final pérdida de tiempo e información útil. De esta manera, la mayoría de modelos predictivos no tienen ni esa capacidad de razonamiento ni ese objetivo de tratar de interpretar de forma autónoma la información que procesan y mucho menos de que la salida de dicho modelo conste de ese contexto y naturalidad requerida. En este escenario, la inteligencia artificial generativa y concretamente los sistemas de agentes inteligentes han hecho de este problema una oportunidad, la cual considero que es más que necesario expresar.

1. Justificación del proyecto realizado

Este proyecto surge de la unión entre una motivación personal y bajo mi punto de vista, por una necesidad tecnológica real. La idea principal era estudiar y desarrollar un proyecto modo prototipo con agentes inteligentes basado en inteligencia artificial generativa. Dicha idea se fundamentó durante las prácticas curriculares del máster en la empresa XRF.ai, en un contexto en el cual, los compañeros de su departamento de inteligencia artificial estaban

interesados en esta nueva línea de investigación, que junto a mis ganas por aprender y motivación por descubrir este subsector dentro de la IA generativa hizo que se pudiera hacer realidad. Además, este propósito coincidía plenamente con un interés personal previo por la tecnología aplicada a casos reales y tangible que lleven al bienestar social y a la protección de la naturaleza y espacios naturales que tan poco valoramos. De esta manera, el proyecto no emergió como un ejercicio tecnológico, sino que también estaba alimentado por los ánimos y contribuir con una causa con impacto directo en la sociedad y en el medio ambiente. Por lo tanto, la elección de este tema responde a una doble motivación, por un lado, a la voluntad de desarrollar una herramienta técnica con utilidad práctica y real en ámbito medioambiental, y el interés por el campo emergente dentro de la inteligencia artificial como son los sistemas de multiagentes, el razonamiento autónomo y la comunicación en lenguaje natural. Por otra parte, la iniciativa se alinea completamente con la filosofía de XRF.ai, una empresa que, operando en sectores de defensa y emergencias, además promueve las soluciones aplicadas al apoyo en decisiones críticas.

Desde una perspectiva personal, este proyecto define de manera perfecta la palabra oportunidad. Una oportunidad para aplicar conocimientos aprendidos a lo largo de este curso en el Máster de Inteligencia Artificial, una oportunidad para profundizar en una rama de la IA generativa como son los sistemas multiagentes y una oportunidad para hacer algo real y contribuir tanto social como ecológicamente. Este proyecto ha sido impulsado en un primer lugar por un alto grado de implicación debido a mi ambición por el aprendizaje pese a la complejidad inicial. No es ningún secreto que tanto los profesionales del departamento de IA de la empresa como mi propio tutor de prácticas no tenían experiencia previa con este tipo de herramientas. Bien es cierto, que a todos nos entusiasmaban y habíamos leído unas cuantas veces noticias al respecto teniendo únicamente una impresión teórica. Por lo tanto, se ponía sobre la mesa un reto, comenzando por un trabajo de capacidad de adaptación y de entender y estudiar a partir de otros proyectos en GitHub o Huggingface. Esto nos permitió, dar pasos firmes en áreas complejas como el

razonamiento de agentes, búsqueda y tratamiento de datos meteorológicos, cálculos avanzados de probabilidades y alertas.

En cuanto al punto de vista académico, considero un trabajo que demuestra la utilidad y escalabilidad que puede llegar a tener la inteligencia artificial generativa. Todos sabemos al ritmo que la Inteligencia artificial avanza y se desarrolla y más recientemente la sección que más está progresando es la de inteligencia artificial generativa. Esto supone que un máster oficial como es este le sea prácticamente imposible estar completamente al día, por lo que mientras otros conceptos de machine learning y deep learning están más asentados los relacionados con al IA generativa no están tan presentes por lo que, aunque contaba con una base, a partir de ahí han sido todos conceptos completamente nuevos. A diferencia de estás otros enfoques de la IA basados en modelos predictivos, esta nueva vertiente ofrece una capa de razonamiento autonomía que amplía las posibilidades de comunicación en lenguaje natural entre usuario y modelo, abriendo nuevas líneas de investigación como estos agentes y siendo capaces de dar muchas soluciones.

Actualmente, la IA generativa está revolucionando y transformando la tecnología permitiéndonos acelerar procesos de toma de decisiones, reducir el error humano y tener sobre todo una capacidad de predicción a tiempo real y en un tiempo récord. Considero que no solo es importante obtener un buen resultado y que sea útil, sino que llegue esta información a tiempo.

2. Presentación del proyecto y sus contenidos

El principal problema que aborda parte de una necesidad cada vez más presente en los sistemas de gestión ambiental, es decir, falta de herramientas capaces de analizar de con autonomía y en tiempo real grandes volúmenes de datos y posteriormente, dar resultados útiles y entendibles. En el caso específico de los incendios forestales, la persona encargada de la toma de decisiones no puede dividirse y estar en varios sitios a la vez, lo que conlleva tener a personas o dispositivos abarcando diferentes áreas y que, a la hora de tomar una decisión, tener que recabar toda esta información realizar cálculos

pertinentes e interpretar los resultados. Esto sin hablar de los errores en la comunicación entre diferentes entidades o usuarios o que no estén sincronizados en tiempo real y con precisión estas diversas fuentes de información. Hay muchas cosas que controlar y que estudiar antes de tomar una decisión y nuestro compromiso es con hacer esta gestión multitarea de manera mucho más sencilla accesible y rápida. Las distintas variables que entran en esta gran ecuación es información de meteorología y geografía en tiempo real, lo que incluye temperatura, humedad, viento, relieve, entre otras. Estos parámetros varían constantemente y con un carácter no lineal lo que contribuye a una dificultad extrema a la hora de usar modelos con técnicas más tradicionales. Aunque cada vez es más lata la disponibilidad de datos en tiempo real, la mayoría de los sistemas actuales, siguen siendo rígidos y con escasa capacidad de predicción y razonamiento ante eventos imprevistos.

Esta limitación muestra un vacío tecnológico que motiva a la tecnología y a mí personalmente para la realización este Trabajo fin de Máster. Tener la capacidad de integrar inteligencia artificial generativa en agentes inteligentes capaces de manera entendible y adaptada mediante mapas geográficos el riesgo de incendio, la probabilidad de ignición y cualquier atributo meteorológico requerido hace que en mi opción sea la pena de recursos humanos y materiales. Por lo tanto, la propuesta se basa en el desarrollo de una aplicación denominada Agente de Emergencias, diseñada para actuar como un asistente inteligente que combina cualidades para una comunicación a través de lenguaje natural, un razonamiento autónomo y un conjunto de diferentes herramientas analíticas a su alcance. Toda esta combinación bien orquestada resulta en la finalidad de ofrecer una solución funcional y demostrativa que permite evaluar un amplio abanico de aspectos de consideración que ayudarán y asistirán a la persona o entidad responsable encargada de la toma de decisiones.

XRF.ai, además de trabajar en defensa donde coordina operaciones estratégicas y acciones de primera línea y en industria donde proporciona conocimiento de la situación en tiempo real y coordinación impulsada por IA, trabaja para las emergencias mejorando el conocimiento de la situación,

agilizando los procesos de tomas de decisiones y optimizando la asignación de recursos, reduciendo el impacto de las emergencias en las poblaciones afectadas. Este trabajo responde por lo tanto a una necesidad compartida: estudiar como los modelos de IA generativa, concretamente los sistemas multiagentes, pueden actuar como núcleo de razonamiento y orquestación dentro de sistemas de monitorización y análisis de riesgos reduciendo costes y tiempo mientras se mejora la capacidad de anticipación y precisión.

El propósito del Agente de Emergencias es crear un prototipo funcional, con el que se permita demostrar un doble propósito. Por un lado, demostrar la viabilidad técnica del uso de agentes inteligentes como orquestadores de un conjunto de herramientas que combinan fuentes de datos en tiempo real y calculo d análisis complejos. Por otro lado, validar su aplicabilidad en un entorno y contexto real. Todo el flujo se gestiona a través del agente ReAct (Reason + Act), que permite entender la solicitud del usuario, planificar que pasoso y que herramientas usar y en qué orden y utilizar sus resultados para elaborar una respuesta completa final. Una de las herramientas integra información procedente de la API Open-Meteo, la cual proporciona todo tipo de datos en relación con viento, temperatura, humedad o estado del cielo. También hay herramientas de geocodificación localizar en coordenadas ubicaciones dadas con lenguaje natural u otra herramienta para la generación de puntos de coordenadas alrededor de un punto geográfico. Además, incluye herramientas de cálculo que usan resultados de las anteriores herramientas para el cálculo de la probabilidad de ignición o el riesgo de incendio según la alerta. Por último, la herramienta más compleja se basa en la generación de heatmaps donde se pueden representar en un mapa real y en tiempo real distinto parámetros.

El uso del framework LangChain de código abierto como marco de desarrollo proporciona la capacidad de concretar el modelo de IA generativa alojado en Azure OpenAi con los diferentes módulos del sistema. La comunicación de todos estos elementos permite desde geocodificar una ubicación dada hasta generar un mapa de calor que representa el nivel de riesgo interpolado en el territorio que nosotros escojamos. Es más, la interacción de usuario se realiza a través de la interfaz web creada con Streamlit, lo que facilita de manera

didáctica la experiencia con el modelo. Gracias a esto, obtenemos el tipo de mapa que deseemos en el lugar que escojamos y con el nivel de detalle y amplitud del área que prefiramos. El resultado ofrece una comprensión semántica, cálculo analítico y visualización geoespacial, en un proceso automatizado y en tiempo real.

El desarrollo de este sistema requirió de habilidad en el desarrollo de código en el lenguaje de programación de Python y se necesita una integración coherente de componentes que van desde los modelos generativos hasta módulos de visualización pasando por bases de datos de referencia y muchas librerías que debes conocer y saber implementar. La combinación de tantos registros diferentes es vital para la versatilidad de la inteligencia artificial generativa cuando se utiliza no solo como modelo de lenguaje sino como motor de razonamiento. En el marco académico este progreso evidencia la evolución de los modelos de IA desde las herramientas más clásicas predictivas a través de estadística avanzada hacia sistemas cognitivos capaces de interactuar tanto con humanos como con los datos.

La relevancia práctica de este proyecto no es concreta y no solo se centra en este caso de uso en específico. El compromiso es con que se extienda más allá y se extrapole no solo a la predicción de incendios forestales sino a otros riesgos ambientales como inundaciones, deslizamientos, terremotos o tormentas eléctricas. El agente puede modificarse y cambiar base de datos y orientando algunas herramientas puede dar lugar a otro agente experto en otra materia. Tanto es así, que incluso esta misma idea y estructura se puede extrapolar a cualquiera otro uso, me refiero a que un sistema de orquestación coherente y cuyas conexiones con sus módulos de herramientas sea correcto y fluido puede abarcar desde la creación de un agente par aun bufete de abogados hasta la administración y gestión de personal y cuentas y cualquier otra cosa que te imagines, es por ello que mi motivación recae de manera considerable en el potencial de escalabilidad y reutilización que previsualizo con este marco operativo.

Desde un punto de vista metodológico, el proyecto combina enfoques de ingeniería de software, aprendizaje automático y diseño de orquestación de

agentes. No solo se buscó el resultado de un heatmap programable, sino que se buscó la interpretabilidad y poder preguntarle a través de un input en lenguaje natural cualquier cosa que pueda estar a su alcance con las herramientas. La arquitectura distintiva es el agente ReAct que se diferencia de los modelos predictivos convencionales en que no se limita a obtener un resultado o predicción, sino que también entender cómo con qué pasos se ha alcanzado dicha conclusión. En términos de impacto, el sistema ofrece una nueva manera de mejorar la eficacia de la prevención de planificación ante incendios, facilitando la información útil al alcance de los responsables de emergencias.

En cuanto a la organización de los documentos, el Trabajo Fin de Máster se estructura mediante 6 capítulos que reflejaran el desarrollo, análisis y validación del sistema junto a una demostración final. Después de esta introducción, el capítulo dos presenta los objetivos generales y específicos que alimentan la ambición de este trabajo tanto de un carácter técnico como de investigación. El capítulo 3, corresponde al Marco Teórico dónde se describirá la información científica relevante mediante información actualizada y citas. El capítulo 4 que hace referencia al Marco Metodológico describe de manera detallada el método seguido en el desarrollo del proyecto, la naturaleza de la investigación y las fases del proceso. Se expone la población objeto de estudio, los objetivos de la investigación, los instrumentos de evaluación utilizados, el diseño experimental aplicado y el modelo de análisis de datos. El Capítulo 5, titulado Presentación de análisis, resultados y discusión, recoge los resultados obtenidos tras la implementación del sistema y del trabajo de campo realizado. Los subapartados que constan en su interior tocarán temas como el análisis descriptivo de la muestra, los resultados obtenidos y la discusión de los resultados que interpreta su significado y evalúa su utilidad, tanto desde el punto de vista técnico como desde su potencial para la aplicación práctica. Finalmente, el Capítulo 6 se basa en las Conclusiones y más concretamente en los subapartados de Limitaciones, Prospectiva del trabajo además de las consideraciones finales. En este último bloque se vuelven a mencionar los objetivos iniciales para valorar en qué medida se han completado y se reflexiona sobre lo aprendido durante todo el desarrollo del proyecto. El

capítulo concluye con una reflexión final sobre las competencias adquiridas a lo largo del Máster y el potencial de la inteligencia artificial generativa.

Capítulo 2: Objetivos del TFM

La definición y explicación de objetivos constituye quizás el componente más esencial dentro del desarrollo de cualquier proyecto o trabajo complejo ya que no solo te permite saber la dirección en la que orientarlo, sino que también un propósito que te permite enfocarte y motivarte con metas. En este caso, considero que los objetivos de mi Trabajo Fin de Máster orientan de forma definida y precisa los procesos metodológicos y técnicos. Este capítulo tiene como finalidad exponer de manera clara, concisa y estructurada tanto el objetivo general que engloba todo el trabajo como aquellos objetivos más específicos a partir de los cuales se logrará obtener más detalle y explicación del propósito fraccionándolos en varios objetivos derivados sobre las metas a alcanzar con este proyecto.

En este marco, los objetivos además de señalar las acciones a ejecutar reflejan la intención académica y aplicada la investigación. Dentro de la inteligencia artificial generativa, la redacción de objetivos permite que al final del trabajo sepamos conocer la viabilidad de construir un sistema funcional a partir de agente inteligentes con capacidad de razonar, planificar y tomar acción ejecutando tareas complejas y en tiempo real. Por lo tanto, estos objetivos se convierten en la base y cohesión entre el planteamiento de ciertas metas a priori alcanzables y la posterior reflexión según la realidad obtenida en los resultados.

El capítulo se organiza y estructura con dos apartados bien distinguidos. En un primer lugar, se encuentra el objetivo general, que comprime y concentra el propósito global de este Trabajo Fin de Máster en una breve definición. Por otra parte, y como segundo apartado, se plantean diferentes objetivos específicos a través de los cuales se desglosará este objetivo general anteriormente comentado llegando a una explicación más detallada. Este bloque tiene una estrecha relación con los bloques posteriores de marco

teórico y metodológico, y una mejor forma de estar en el apartado de conclusiones. Como conjunto, estos objetivos ofrecen una visión completa de las metas que constituyen el alma de esta investigación y guían el desarrollo técnico en la dirección deseada.

1. Objetivo general

Desarrollar un sistema con un agente orquestador basado en inteligencia artificial generativa que permita demostrar su viabilidad y eficiencia operativa como herramienta funcional en el análisis de riesgos de incendios.

2. Objetivos específicos

Estudiar los principios de la inteligencia artificial generativa en el paradigma de los sistemas multiagentes para encontrar la mejor arquitectura y framework para el desarrollo de este sistema con un agente orquestador.

Analizar e investigar sobre la inteligencia artificial generativa y su integración con sistemas multiagentes es considerado un punto esencial para la posterior planificación y desarrollo de este trabajo. Este punto permite comprender diferentes conceptos técnicos y complejos que no se conocían previa o simplemente de oídas o incluso otros con los que si se ha trabajado pero que conviene tenerlos aún más presentes y a la orden del día para poder obtener a través de ellos una correcta implementación. Alcanzaremos un conocimiento profesional tanto en modelos de lenguaje de gran escala, como en el amplio abanico de posibilidades dentro del mundo de los agentes de orquestación y razonamiento. Asimismo, este análisis ayuda a identificar los puntos fuertes y desventajas de la IA generativa en este aspecto en comparación a enfoques más tradicionales basados en aprendizaje supervisado. Además, toda esta información nos conviene al crear una base de literatura científica y técnica a partir de la cual orientar posteriores decisiones de diseño y desarrollo que se mostrarán más adelante.

Diseñar la estructura funcional del sistema, definiendo módulos y flujos de información necesarios que permiten la interacción entre el agente y las entidades a su alcance como son el usuario, las herramientas y las fuentes de datos.

Diseñar la arquitectura técnica del Agente de Emergencias, tiene como finalidad construir la estructura lógica de este sistema y que permita de la manera más eficiente posible la comunicación entre las entidades implicadas. La arquitectura debe tener una disposición versátil ya que a partir de la cual se puedan añadir eliminar o modificar herramientas sin alterar en exceso el comportamiento general del sistema y sin tener que rehacer todo el proceso desde el inicio. Se busca que la estructura final sirva de ejemplo y de base para implementarse en un caso de uso distinto a del análisis de riesgo de incendios o a la mejora o actualización de este. El diseño se compone de una capa de razonamiento sostenida en el modelo generativo a través del agente ReAct (Reason + Act), una capa de procesamiento, cálculo y visualización posterior en la que aparentemente las diferentes herramientas están al mismo nivel, pero siempre teniendo en cuenta que unas dependen de otras y una capa final que se usará como interfaz para la interacción del usuario. Se pretende implementar una estructura flexible, polivalente u eficiente que asegure la funcionalidad del sistema y una comunicación entendible para el usuario.

Implementar el agente orquestador mediante el uso del framework correspondiente de IA generativa, integrando los módulos de geocodificación, análisis meteorológico, cálculo de probabilidades y riesgos y generación de visualizaciones interactivas.

Implementar el agente orquestador es el verdadero reto que supone materializar el diseño teórico en un sistema operativo capaz de razonar de forma autónoma a partir de input del usuario en lenguaje natural da dado.

Lo que significa entender la solicitud y tener claro que herramientas necesita y en qué orden de las que están a su alcance para poder obtener toda la información necesaria para elaborar una respuesta lo más completa posible. Para ello, se ha utilizado el framework de LangChain que es el que contiene este tipo de arquitectura de agente en concreto y Azure Open AI como núcleo cognitivo del agente. Como bien se ha comentado anteriormente, la siguiente acción recae en la integración de todas estas herramientas que se necesitan para obtener toda la información necesaria para desempeñar la función de asistente como analista de emergencias por riesgo incendios. Una de las partes más esenciales de este agente es la programación de su System Prompt. Aquí es donde se le explica, a través de lenguaje natural también, que herramientas tiene a su alcance, para que sirven y sobre todo que valores necesita cada herramienta como entrada y qué devuelve como salida. Al fin y al cabo, partes esenciales para cualquier entidad ya sea humana o no para resolver una acción: saber que herramientas tiene a su alcance, para qué son útiles y como se usan. A partir de esta disposición, el agente logra coordinar y gestionar sus recursos a través de las llamadas que estime necesarias a cada herramienta y en el orden oportuno.

Comprobar el funcionamiento operativo del sistema y evaluar sus respuestas para que ofrezcan resultados coherentes, precisos y entendibles de cara al usuario.

Comprobar el desempeño funcional del sistema es una parte fundamental en el proyecto dado que es prácticamente imposible que algo salga con una calidad certera en un primer intento. Al final, siempre hay que pasar por esa fase de prueba en la que analizas los resultados para evaluar su calidad y a partir de ahí considerar que cambios realizar y qué modificar en la etapa anterior. La evaluación experimentada en este trabajo fin de máster consta del análisis de la consistencia de los mapas generados, que los cálculos complejos pertinentes estén correctos y en caso de que no, tratar de averiguar

si la fuente del error proviene de los datos obtenidos a partir de las diferentes fuentes de información o de la herramienta de cálculo en sí. Además de esa evaluación, también se estudia cuantas llamadas hace a cada herramienta en que orden para tratar de entender su comportamiento y si da pasos redundantes o innecesarios y se comprueba que los recursos tantos de tokens como de tiempo empleados son coherentes. A través de todo este proceso, se analiza la capacidad resolutive global del proyecto de manera real lo que ayuda a ir formando conclusiones sobre limitaciones y próximos aspectos a mejorar.

Examinar y llegar a conclusiones con respecto a posibles líneas de mejora, posibles limitaciones y el potencial de escalabilidad del sistema en futuras aplicaciones.

Estudiar estos aspectos constituye un paso primordial de cara a una proyección futura y su relevancia dentro del ámbitos e inteligencia artificial aplicada. Este objetivo busca llegar a conclusiones certeras y fundamentadas sobre todas las capas y módulos que componen este sistema. Reconocer estos factores y comprender tanto sus fortalezas como sus flaquezas hacen que se enriquezca esta investigación y en definitiva las conclusiones de este Trabajo fin de máster. No obstante, sin este análisis no podríamos argumentar sobre la capacidad de polivalencia y escalabilidad de este tipo de arquitectura en otros aspectos y casos de uso diferentes. En conjunto, este objetivo permite cerrar esta línea de investigación y desarrollo aportando de manera rigurosa y fundada conclusiones válidas para el posterior desarrollo d este mismo proyecto a un nivel profesional o cualquier otro aplicando la base de esta arquitectura teniéndolo como referencia.

Una vez definidos los objetivos que orientan el presente trabajo, en el siguiente capítulo se abordará el marco teórico que trata los conceptos base de este proyecto.

Capítulo 3: Marco Teórico

El presente capítulo constituye el marco teórico que fundamenta el diseño e implementación del agente de emergencias mediante la explicación de los conceptos teóricos teniendo de referencia la inteligencia artificial generativa como tema principal. En un primer momento se planteó la posibilidad de encontrar algún proyecto que englobe tanto el tema de los incendios forestales como el uso de tecnología de sistemas multiagente con IA generativa. Sin embargo, debido a la novedad de esta tecnología, no existen iniciativas previas que permitan tomarla como referencia para este caso de uso concreto. Por ello, se han buscado fuentes y estudios específicos para cada una de las secciones que conforman este proyecto. La primera sección abordará el tema de agentes como núcleo cognitivo para el razonamiento y la orquestación de manera que constará de un análisis sobre la construcción de estos haciendo énfasis en diferentes frameworks estudiados y los que finalmente hemos elegido para la implementación y los motivos de cada una de las decisiones que se han ido tomando. El capítulo continúa con la exposición de los conceptos de meteorología necesarios para la evaluación del riesgo de incendio, por lo que realizaremos un análisis para mostrar la forma en la que se ha obtenido las variables de temperatura, humedad relativa, viento y estado del cielo a través de APIs meteorológicas de acceso programático y actualización frecuente para la obtención de los datos requeridos en tiempo real. Seguidamente, se revisarán los procesos de geocodificación que han sido necesarios para traducir consultas en lenguaje natural por parte del usuario a coordenadas útiles operacionalmente hablando. A partir de ello, estudiaremos los métodos utilizados para la generación de mallas geográficas y muestreos espaciales próximos, lo que nos ha permitido estructurar toda la información obtenida en una zona geográfica correctamente medida. A continuación, se analizarán las fórmulas y los cálculos avanzados para la estimación de la probabilidad de ignición y del estado de alerta según el riesgo de incendio explicando el origen de las fuentes de información y la propia metodología implementada para la realización de dichos cálculos. Además, se describen los fundamentos

de la interpolación espacial y la construcción de mapas de calor donde abarcaremos desde criterios de elección de dichos métodos hasta la forma de implementar estas técnicas con toda la información anterior para que esté coordinada a la perfección. Por último, se explicará la visualización de resultados a través de Streamlit como capa de interfaz para la interacción entre del usuario con el agente y la navegación visual a través de los diferentes resultados.

Todos estos temas son los que constituyen el marco teórico y cuyo objetivo es fundamentar los pilares teóricos de este proyecto para posteriormente tratar en el siguiente capítulo con el marco metodológico partiendo de una base teórica sólida y consolidada.

1. Agente orquestador

A continuación, se va a desarrollar el subapartado del agente orquestador. Sin embargo, no solo se ha estudiado el agente ReAct, sino que antes de tomar la decisión de usar este tipo de agente, se ha analizado el framework de Smolagent y en concreto los agentes ToolCallingAgent y CodeAgent. Por lo tanto, vamos a explicar en qué se basan estos dos agentes en un primer lugar para poder situar el contexto y desarrollar los motivos de esta decisión dado que estos agentes también son muy prometedores y potentes técnicamente y merecen la mención.

ToolCallingAgent

Hugging Face. (2024). *Tools – Smolagents Documentation (v0.1.3)*. Hugging Face Docs. <https://huggingface.co/docs/smolagents/v0.1.3/tutorials/tools>

El agente ToolCallingAgent, en lugar de resolver un problema mediante razonamiento lingüístico se encarga de comunicarse de manera programática herramienta externa para obtener una respuesta de ellas y así avanzar en la tarea. Es un agente muy interesante en términos operativos ya que a efectos

prácticos es capaz de controlar un conjunto de funciones definidas para obtener un resultado posterior complejo. Las llamadas herramientas se escriben en formato JSON con argumentos estructurados, lo que implica su capacidad para resolver y tratar este tipo de inconvenientes de manera eficaz. No obstante, la primera desventaja que se observa desde un primer instante es que como así se demuestra en la fuente oficial en Hugging Face. (2024). *Tools – Smolagents Documentation*, el agente necesita un system prompt muy detallado, no tiene la capacidad de razonar y tienes que hacerle entender en este prompt que es exactamente lo que quieres que haga, cuáles son los pasos que tienes que dar y de manera muy explícita y explicativa. Además, esto supone un contexto muy amplio en el momento en el que el flujo de sistema que se esté creando utilice varias herramientas y haya diferentes supuestos que el usuario a través de la consulta puede solicitar. Por tanto, tras implementar un pequeño prototipo con este agente, da la sensación de que se está realizando un programa complejo programable corriente simplemente que en vez de tener bucles y código para los diferentes decisiones y condiciones se obtiene un bloque de texto que incluso se vuelve menos claro y complejo que un simple programa con funciones if y while por poner un ejemplo.

Después de analizar este problema el cual planteaba más dudas que soluciones, había una esperanza de que la opinión con respecto a este agente pudiera cambiar. Esta posible ventaja radicaba en el consumo de tokens ya que algunos estudios como este evidenciaban que el consumo de tokens de este agente era extremadamente bajo y es por ello por lo que resulta muy atractivo. Es cierto que el consumo de tokens era bajo, pero aun así el hecho de un prompt tan detallado suponía que esta ventaja se quedase en un simple comentario teórico, o al menos para este caso de uso.

El estudio práctico se ha llevado a cabo teniendo como referencia el documento al inicio de este apartado. El framework utilizado es Smolagents y gracias a los ejemplos prácticos de dicho documento no ha sido del todo complejo inicializar este agente con un conjunto mínimo de líneas de código ya que el recorrido es guiado y la API de referencia describe de manera exhaustiva cada uno de los procedimientos. Este agente defiende la idea de que el formato JSON es la solución para llamadas estructuradas y heterogéneas que permite

simplificar las comunicaciones entre diferentes nodos dentro del sistema, siendo esta su principal baza. Además, esta biblioteca documenta patrones de orquestación multiagente y prácticas para mejorar la comunicación entre el usuario el LLM y el entorno, lo que era otro aliciente más para que se considerase un candidato para ser el núcleo cognitivo de este proyecto.

CodeAgent

Hugging Face. (2024, enero 30). *Introducing Smolagents: Simple Agents That Write Actions in Code*. Hugging Face Blog. <https://huggingface.co/blog/smolagents>

El agente conocido como CodeAgent representa una evolución de los sistemas de inteligencia artificial autónomos permitiendo que el propio modelo de lenguaje no solo decida qué acciones ejecutar, sino que genere su propio código fuente y lo ejecute en tiempo real siendo esta técnica parte de su proceso de razonamiento. Según la documentación oficial de *Hugging Face* (2024), el enfoque de *Smolagents* para este agente es que genere códigos de Python que implementen la solución y que genere el código dentro de un entorno controlado. De esta forma el agente combina el lenguaje natural suministrado por parte del usuario y una planificación y programación propia ampliando exponencialmente la capacidad para la resolución de tareas complejas. Este agente se basa en tres componentes principales: el modelo generador, el intérprete seguro y el conjunto de herramientas. El modelo, que suele ser un LLM avanzado, se encarga de formular el plan de acción y escribir las instrucciones del código. Por otro lado, el intérprete, aislado en un entorno sandbox, ejecuta el código de manera controlada y captura la salida. Por último, se trata del conjunto de herramientas que la gente puede invocar para lograr resolver cuestiones más concretas y específicas.

Como se puede observar, este agente es más que interesante dada su capacidad de generalización y adaptabilidad. Se trata de una arquitectura con un potencial impresionante capaz de ser implementado bajo prácticamente cualquier caso de uso gracias a su polivalencia flexibilidad y capacidad

resolutiva de alto nivel. Sin embargo, la potencia del agente CodeAgent no siempre iba a tener ventajas de su lado. En cuanto a los principales problemas que conllevaría su implementación en un trabajo como el que se presenta en este TFM, es en primer lugar el riesgo en la seguridad. El hecho de que se necesite una ejecución dinámica de código repercute en retos operativos con posibilidad de llamadas no deseadas con redundancia y errores que se escapan a la hora de la ejecución. Además, puede desembocar en un consumo de tokens excesiva, que es lo primero que se observó cuando se realizó un pequeño prototipo para experimentar su capacidad y si realmente era útil en este caso de uso. La diferencia con respecto al ToolCallingAgent y al ReAct agent es abrumadora consumiendo varias veces más tokens que los otros dos agentes. Otra limitación o más bien complejidad por parte de este sistema, recae en que como bien se puede observar en la cita, su programación es de alto nivel al tener que usar técnicas de seguridad complejas como es sandbox para poder controlar el código generado por parte del agente lo máximo posible. En el caso anterior con ToolCallingAgent que ya parecía un reto la redacción del system prompt, el nivel de programación para realizar e implementar este agente es aún más difícil aun siguiendo el documento al pie de la letra.

Por último y como argumento clave para descartar esta opción, resulta que a efectos prácticos y la principal cualidad de este agente no es otra que su potencial y su mayor libertad creativa, No obstante, en el flujo de trabajo y de herramientas que vamos a usar no requiere de un agente orquestador con tanta capacidad técnica, ya que esta carga compleja se derivará a cada una de las herramientas y la función específica del agente orquestador recaerá como su propio nombre indica, en dirigir, razonar y orquestar las diferentes decisiones y herramientas para el resultado más óptimo posible. Por lo tanto, se considera que la complejidad de esta arquitectura y su consumo no compensa lo que puede aportar.

ReAct Agent

LangChain AI. (2024). *How to Create a ReAct Agent from Scratch (Functional API)*. LangGraph Documentation. <https://langchain-ai.github.io/langgraph/how-tos/ReAct-agent-from-scratch-functional/>

Antes de comenzar con la definición explicación y análisis del propio agente ReAct, se va a exponer una introducción del framework al que corresponde para partir de una base sólida y contextualizada ya que tanto Lanchain como Landgraph representan actualmente dos de los frameworks más avanzados para la construcción de agente inteligentes. Lanchain, surgido en 2023, se ha consolidado como un entorno de referencia de flujo de razonamiento con modelos de lenguaje a gran escala ya que permite conectar LLMs con herramientas, base de datos, APIs y canales de procesamiento a través de un código estandarizado reutilizable y altamente configurable. Su arquitectura, como su propio nombre indica, está basada en chains y agents lo que posibilita la interconexión de diferentes módulos en sistemas complejos y variados como se da en este trabajo. Por otro lado, Landgraph representa una evolución de Lanchain ya que da un paso más en la orquestación donde cada uno de sus nodos representa una etapa de razonamiento o acción y las transiciones permiten encontrar el flujo de decisión del agente. Mejora la reproducibilidad y la estabilidad en la ejecución de tareas complejas además de facilitar su visualización. En conjunto, ambos frameworks proporcionan un entorno seguro escalable y de código abierto que impulsa el desarrollo de agentes autónomos de manera actualizada y yendo a la última, lo que justifica que sea muy atractivo utilizarlos en este TFM como infraestructura base del agente ReAct.

El agente ReAct, acrónimo de *Reasoning + Acting*, Representa una arquitectura avanzada de agente inteligente que ya tiene integrado esa capacidad de razonamiento sobre el problema, la capacidad de elección de herramientas a su alcance y la evaluación de la respuesta para reconocer si necesita más información por parte de algún módulo. Según la guía de LangGraph (LangChain AI, 2024), el agente ReAct implementa un bucle en el que, ante una solicitud o consulta por parte del usuario, el modelo del

lenguaje analiza la situación (Thought), posteriormente decide qué herramienta invocar (Action), la ejecuta y por último recibe la salida (Observation) y vuelve a reflexionar (Thought). Este flujo de trabajo permite combinar reflexión tanto del input por parte del usuario como de la respuesta final para poder entregar una resolución que se adecue perfectamente al usuario y a sus posibilidades a partir de la herramienta que tiene a su alcance.

El documento citado muestra cómo construir este tipo de agente desde cero. Primeramente, define un estado que contiene elementos como el historial de mensajes del usuario o posibles datos intermedios generados por herramientas. Posteriormente, se definen y se presenta a la gente un conjunto de herramientas y, por último, se les entrega un system prompt que básicamente contiene la definición y funcionalidad de cada una de las herramientas y qué necesita cada una de estas como input y qué obtienes como output. Al final, no necesitas darle un contexto complejo y explícito como puede ser en el caso del ToolCallingAgent, sino que basta con una breve descripción para que tenga conocimiento de qué tiene a su alcance. Descripción que en el ámbito real cualquier persona necesita para poder realizar cualquier tarea compleja. Me refiero a que una persona por muy capaz que sea, antes de realizar por ejemplo un trabajo de fontanería necesitas saber qué es un martillo o una llave inglesa, con una breve indicación es capaz de intuir las posibilidades de esa herramienta y de saber cuándo utilizarla, es simplemente darle un contexto lógico para que sea capaz de usar esa herramienta sin nada de bucles ni de condiciones dentro del mensaje para eso tiene esa capa de pensamiento y razonamiento y es lo que lo distingue del ToolCallingAgent.

La ventaja principal del enfoque ReAct, es el uso iterativo del flujo de trabajo mencionado anteriormente: Thought-Action-Observation. Esto nos permite asegurarnos de un agente puramente desarrollado para orquestar y dirigir, sin embargo, esta no es la única ventaja que presenta. Es cierto, que no tiene la capacidad de generar código por sí mismo y no tiene esa capacidad tan flexible para poder realizar tareas complejas a ese alto nivel como el agente CodeAgent. Sin embargo, eso significa que no hereda una complejidad de código extremo, sino que es incluso más sencillo que el ToolCallingAgent,

muy fácil de implementar y además muy estandarizado gracias al framework del Langchain. El uso de este agente eficaz y sencillo hace que su consumo en tokens sea extremadamente bajo, comparado con los anteriores, varias veces inferior al de CodeAgent y levemente inferior al ToolCallingAgent. Solamente puede haber un caso en el que se presente con agent como una mejor opción y es aquel en el que su caso de uso sea extremadamente complejo y se necesite que el agente haga cálculos a través de código asumiendo el riesgo de inseguridad que eso conlleva, sin embargo, en este proyecto no es necesario estos cálculos avanzados, los que hay se hacen a través de herramientas y además obtenemos una ventaja que radica en una seguridad abrumadora frente al CodeAgent.

Por estas razones y tras evaluar los tres tipos diferentes de agentes, dado los requisitos que presenta el sistema de análisis de riesgo de incendios y la complejidad de este agente de emergencias, implementar el modelo ReAct como agente orquestador principal es una decisión que favorece la fiabilidad, razonamiento y adaptabilidad en este trabajo.

2. API Meteorológica para la obtención de datos en tiempo real

El acceso a información meteorológica precisa y actualizada en tiempo real es un componente más que esencial en un sistema de predicción de incendios como es nuestro caso de uso. Básicamente, todos nuestros cálculos futuros parten de esta fuente de datos como información base para el desarrollo posterior de todas las herramientas e implementaciones. En este proyecto, la obtención de los parámetros meteorológicos necesarios se ha realizado mediante la API pública Open-Meteo que consta de un servicio especializado en datos en tiempo real a partir de coordenadas geográficas. La documentación técnica de Zippenfenig ha servido de referencia para entender el funcionamiento de este tipo de APIs y conocer cuál es la que más se adecuaba a nuestras necesidades. Además, ha conseguido que obtengamos un conocimiento amplio sobre las diferentes posibilidades dentro de este marco y que no solo se obtenga información referente a las necesidades específicas de este proyecto. Es por ello por lo que se selecciona esta fuente de información

que nos ofrece una visión más amplia y enriquecedora de esta herramienta tan útil y necesaria en tantos trabajos y estudios. Esta API ofrece un modelo de acceso libre a la información meteorológica basada en predicciones numéricas globales con la capacidad de generar repuestas en un formato JSON, algo que dada la naturaleza de nuestro agente y de cómo se quiere orientar el intercambio de información entre los diferentes nodos del modelo, se considera un requisito fundamental. Se obtienen los parámetros de temperatura, humedad relativa, velocidad del viento y estado del cielo, todos ellos más que imprescindibles de una manera u otra en las de diferentes herramientas del proyecto. Su arquitectura está orientada al acceso mediante solicitudes HTTP, lo que permite una comunicación directa en aplicaciones de análisis o en sistemas autónomos de toma de decisiones como es este caso.

El agente de emergencias implementa una herramienta específica encargada de realizar estas consultas a partir de las coordenadas de latitud y longitud obtenidas en el proceso de geocodificación. Mediante peticiones GET estructuradas, el sistema es capaz de obtener los valores meteorológicos en tiempo real filtrando y trayendo únicamente los parámetros que son de utilidad. Este enfoque permite que este proyecto cobre sentido dado que carecería del mismo en caso de que los datos obtenidos a través de esta herramienta no sean actualizados, lo que desembocaría en una aplicación obsoleta y poco capaz en el sentido de la eficiencia.

La decisión de utilizar Open-Meteo frente a otras alternativas comerciales radica en las principales ventajas que otorga, como su carácter abierto y gratuito. A diferencia de APIs como *OpenWeatherMap* o *AccuWeather*, que requieren licencias o limitan el acceso mediante cuotas de uso, Open-Meteo permite realizar las consultas que desees sin requerir autenticación además de ofrecer transparencia total en las fuentes de datos utilizadas. La combinación de todos estos aspectos son los motivos que hacen fundamental esta decisión. Los datos obtenidos tienen una frecuencia de actualización habitualmente de una hora lo que para el objetivo de este proyecto de ser una demo o un prototipo se considera un periodo de tiempo más que razonable, aunque para un futuro o una implementación profesional hará falta una actualización considerablemente más precisa.

Por todos los motivos mencionados anteriormente, no es de extrañar que esta API se convierte en una herramienta fundamental en el proyecto para vincular los datos meteorológicos con el resto de las herramientas del trabajo. Este componente constituye, por tanto, que toda la implementación tecnológica a través de la inteligencia artificial generativa provenga de un contexto real y una fuente fiable a partir de la cual poder desarrollar con garantías resultados futuros precisos.

3. Procesos de geocodificación y obtención de coordenadas

Mocnik, F.-B., Zipf, A., & Raifer, M. (2017). *The OpenStreetMap data model and its application in geocoding and routing*. ISPRS International Journal of Geo-Information, 6(2), 54. <https://doi.org/10.3390/ijgi6020054>

La geocodificación forma parte de otro de los procesos fundamentales dentro del conjunto de herramientas de este trabajo. Su objetivo es transformar una descripción textual de una ubicación, como puede ser el nombre de una ciudad, un municipio o cualquier sitio de interés en coordenadas geográficas expresadas en latitud y longitud. De esta manera, conseguimos que esta herramienta logre traducir las consultas en lenguaje natural por parte del usuario hacia valores geográficos operativos, lo cual, será fundamental para todas aquellas consultas en las que el prompt no indique formato en coordenadas. Por lo tanto, constituye un paso esencial para la realización del agente de emergencias, siendo así el primer paso dentro del flujo de razonamiento del sistema permitiendo obtener la ubicación espacial del entorno a partir del cual se procederá a realizar los análisis y estudios pertinentes en el futuro.

La implementación desarrollada en el proyecto utiliza la API pública Nominatim, integrada en el ecosistema de OpenStreetMap (OSM), que es una de las plataformas de datos geográficos abierta y más desarrollada del mundo. Tras analizar detenidamente el estudio de Mocnik, Zipf y Raifer (2017), se llega a la conclusión de la arquitectura de OSM permite estructurar y mantener un modelo de datos espacial con alta precisión y posibilita que

servicios como Nominatim actúen como interfaces de búsqueda a partir de las cuales se procese la consulta del usuario y devuelva las coordenadas asociadas al nombre del lugar solicitado. Hay que tener en cuenta que es de vital importancia indicar correctamente y de manera concreta estos lugares geográficos. Se comenta, a modo de aclaración, dado que pese a que el usuario puede ser español y pregunte por un centro geográfico como puede ser la ciudad de Córdoba, automáticamente se piensa que nos referimos a la ciudad de Córdoba en la Comunidad Autónoma de Andalucía, al sur de la península, sin embargo, este agente no tiene ningún tipo de contexto y perfectamente le puede indicar las coordenadas de la ciudad de Córdoba de Argentina. Teniendo presente este error común, en la práctica, este proceso combina tanto mecanismos de normalización lingüística y recuperación semántica como de indexación geográfica, lo que permite que sea una herramienta capaz de obtener resultados con una correcta precisión espacial.

Dentro del proyecto, la herramienta de geocodificación del agente de emergencias implementa este proceso a través de solicitudes HTTP hacia el endpoint de Nominatim, especificando los parámetros de búsqueda en formato JSON. Por lo tanto, el flujo de acción sería que una vez que el agente ha solicitado o ha decidido usar la herramienta de geocodificación, este módulo recibe el fragmento específico del texto proporcionado por el usuario y devuelve una lista de los resultados obtenidos incluyendo el nombre completo de la localización y sus coordenadas en latitud y longitud. Además de este resultado, este módulo también contiene una segunda función que no siempre es utilizada, pero en cierto contexto es realmente útil. Se trata de que en caso de que así lo indique el usuario en el prompt, tiene la capacidad de obtener más coordenadas de diferentes puntos geográficos, un ejemplo fácil de entender sería un mensaje por parte del usuario indicando, por ejemplo, que quiere obtener las coordenadas de la ciudad de Córdoba en Andalucía, España y de sus pueblos más importantes dentro de la provincia. A esto el agente es capaz de procesar la información y dado que usa un motor GPT4-omini, obtiene la información respecto a una serie de pueblos de la provincia de Córdoba. Una vez realizada esta búsqueda se le pasa como input de la manera que hemos mencionado anteriormente esta información, que junto a un parámetro regulador dentro del código que limita en número de resultados

posibles para evitar redundancias y que de pronto no extraiga una lista de 120 municipios, devuelve una lista con por ejemplo nueve municipios más la propia ciudad de Córdoba y sus correspondientes coordenadas en longitud y latitud. De esta manera, se asegura una búsqueda eficiente evitando sobrecargar el flujo de trabajo del agente, pero asegurando una respuesta que cumple con los requisitos solicitados por parte del usuario. En cuanto a un último ejemplo, me gustaría mencionar que en caso de que el usuario formulase una consulta tipo riesgo de incendio en La Palma, isla de Las Islas Canarias, España, el agente es más que capaz de entender que lo primero que necesita es obtener la situación geográfica exacta de la isla de La Palma y recurriendo a esta herramienta para la adquisición de ese dato en forma de latitud y longitud.

El modelo de datos de OpenStreetMap consta de tres estructuras principales; nodos, caminos y relaciones. Cada nodo representa una coordenada geográfica, los caminos agrupan nodos para formar elementos lineales y, por último, las relaciones establecen vínculos semánticos entre distintas entidades. Todo esto permite que traduzca eficientemente los nombres de lugares en coordenadas. Como indica Mocnik et al. (2017), esta organización de datos abiertos ofrece un equilibrio entre la precisión espacial y una capacidad de actualización continua que resulta considerablemente valiosa en un contexto de rápida evolución territorial. Se ha tomado la decisión de trabajar con la API Nominatim debido a que, al ser abierto, gratuito y reproducible, facilita considerablemente la integración en nuestro proyecto sin restricciones comerciales ni de autenticación además de transparencia metodológica. Además, representa a una comunidad activa y en constante crecimiento como se puede observar en el artículo y que, en el caso de uso en entornos de emergencias, es crucial para la obtención de resultados precisos en las predicciones.

Todos estos motivos, evidencian que el módulo de geocodificación mediante la API Nominatim de OpenStreetMap, sea esencial dentro del marco del proyecto proporcionando la correspondencia exacta entre las consultas en lenguaje natural por parte del usuario y las coordenadas geográficas requeridas para una respuesta o información requerida por parte de alguna

herramienta. El modelo OSM, a su vez, ofrece una solidez que respalda esta decisión tecnológica, sentando las bases de una herramienta más que trascendental en el flujo de trabajo de este proyecto junto al resto de módulos que tiene a su alcance el agente.

4. Generación de mallas geográficas y muestreos espaciales

Stål, T., & Reading, A. M. (2020). *A Grid for Multidimensional and Multivariate Spatial Representation and Data Processing. Journal of Open Research Software*, 8(2). <https://doi.org/10.5334/jors.287>

El módulo que vamos a presentar a continuación Forma parte del siguiente paso en el análisis espacial debido a que el objetivo de esta herramienta busca estructurar el territorio en celdas o puntos equidistantes que permitan realizar cálculos sobre distintas variables ambientales teniendo en cuenta su localización geográfica. En este agente de emergencias, en un primer momento deberá obtener a partir del mensaje por parte del usuario el fragmento de texto que indique expresamente el punto geográfico central. Este primer paso se llevará a cabo gracias a la herramienta de geocodificación de la que se ha hablado anteriormente. Una vez que obtenemos las coordenadas geográficas de este punto central es cuando la gente debe entender que el siguiente paso es llamar a esta herramienta con el fin de generar una malla geográfica a partir de dicho punto. Por lo tanto, la herramienta desarrollada reproduce el principio fundamental descrito por Stål y Reading (2020), quienes en este artículo estudian los grid multidimensionales como una forma eficiente de representar y visualizar datos espaciales mediante programación.

El trabajo de Stål y Reading (2020), Expone un módulo en el lenguaje de programación de Python denominado *agrid*, el cual está diseñado para gestionar representaciones espaciales estructurada sobre mallas o rejillas regulares. Su implementación además de permitir construir cuadrículas sobre un espacio geográfico determinado en múltiples dimensiones muestra su capacidad para asociar variables de diferente origen como puede ser ambientales físicas o incluso socioeconómicas. Básicamente, está basado en

un sistema de discretización uniforme del espacio que consiste en dividir una superficie en elementos de tamaño constante manteniendo una relación de distancia entre ellos predeterminada. En este proyecto, el agente de emergencias usa esta herramienta con el mismo propósito que muestra este estudio. el objetivo es generar una cuadrícula de puntos alrededor de una coordenada central que se ha mencionado anteriormente, utilizando dos parámetros ajustables que son: por una parte, la longitud en kilómetros de un lado de la malla (ya que la malla que se va a generar va a ser cuadrada). De esta manera se puede ajustar el área que se quiere analizar en función de los requisitos que haya solicitado expresamente el usuario o según la lógica que queramos aplicar debido a que no es lo mismo representar de manera geográfica la provincia de Madrid que un pueblo de Córdoba. Además, no es necesario tener pensado este valor previamente, ya que podemos tener una de manera predeterminada y cuando veamos los mapas generados poder modificar este parámetro con la interfaz de manera muy sencilla de cara al usuario. Esto último se aclarará y explicará con más detenimiento cuando se exponga el subapartado del módulo de Streamlit. Por otra parte, el otro parámetro ajustable radica en la distancia entre puntos y también se mide en kilómetros. Una vez que se obtiene el área de esta malla, queda por conocer cuántos puntos dentro de la misma se van a generar. Hay que tener en cuenta, que cuántos más juntos se analice, el mapa tendrá más detalle y será de más calidad, aunque paralelamente también aumentará el número de tokens consumidos. Un ejemplo visual, es que el primer parámetro sea de 10 km por lo tanto al ser un cuadrado y tener un lado de 10 km el otro lado también será de 10 km teniendo un área total de 100 km. En caso de que el segundo parámetro, que muestra la distancia entre puntos a analizar, sea de 1 km significa que contaremos con un total de 100 puntos analizar dentro de un área de 100 km².

El procedimiento implementado parte de la fórmula de desplazamiento sobre una esfera, considerando el radio medio de la tierra y aplicando conversiones trigonométricas para calcular lo desplazamiento en grados de latitud y longitud en función de los parámetros anteriores. Este cálculo permite obtener de manera precisa cada una de las coordenadas de cada punto dentro de la malla. La función `offset_coordinates` realiza este cálculo garantizando

que cada punto generado mantenga una relación coherente respecto al centro geográfico. Posteriormente, la función `generate_grid_points` itera sobre intervalo equidistante en los ejes X e Y para construir la cuadrícula dimensional añadiendo cada nuevo punto o nueva coordenada a la lista de resultados. Básicamente, actúa como un generador de rejillas regulares en dos dimensiones que servirá para operaciones posteriores y será la base de los mapas de calor resultantes.

Desde el punto de vista funcional, este módulo que es llamado técnicamente dentro del proyecto como `get_nearby_locations`, además de englobar las funciones anteriormente descritas, tiene la capacidad de creación de una malla alrededor de múltiples puntos base. Esto es útil cuando en consultas en las que el usuario solicite información no solo de un punto geográfico central sino de varios núcleos principales. Gracias a esta última característica, el agente además es capaz una región amplia cómo por ejemplo puede ser la península Ibérica teniendo como puntos principales las ciudades más pobladas o relevantes, como puede ser Madrid, Sevilla, Valencia, Barcelona, Oporto y Lisboa.

La elección de un modelo de malla geográficas regular, ha sido también una idea aconsejada por parte del tutor de las prácticas de empresa, ya que mi primera idea fue desarrollar y seleccionar diferentes puntos a partir de una espiral circular que comenzaba en el centro geográfico que se iba expandiendo. Sin embargo, fue una primera idea y estaba seguro de que iba a haber diferentes soluciones más eficientes. Es por ello por lo que pedí consejo a mi tutor y simplemente me comentó que a él le parecía más lógico esta idea y que investigase sobre ello. A partir de ese momento comencé un análisis exhaustivo que derivó en el resultado de este módulo y el estudio de la propuesta de Stål y Reading como referencia. En definitiva, el sistema adquiere una estructura espacial sólida que repercute de manera directa con otros módulos y con el resultado final de las visualizaciones geoespaciales.

5. Cálculo de probabilidad de ignición y riesgo de incendio.

Fuente: En el apartado final de Anexo, documentación técnica facilitada por la empresa XRF.ai (2025), “Tablas de probabilidad de ignición y peligro de incendio utilizadas por las unidades de emergencia de La Palma”.

A continuación, se va a exponer el marco teórico dentro del módulo del cálculo de la probabilidad de ignición y el cálculo de la alerta de riesgo de incendio. Al final, uno de los objetivos que hemos mencionado anteriormente de este proyecto es demostrar que este sistema tiene potencial para que en el día de mañana desarrollado de forma profesional pueda ayudar como asistente en la toma de decisiones para las emergencias por riesgo de incendio. la empresa XRF, trabaja codo con codo con el sistema de emergencias de la isla de La Palma de la Islas Canarias en España. Es por ello, que se ha recurrido a ellos para saber cómo calculan y qué métodos utilizan para el cálculo de estos parámetros. Como se observa en las imágenes, nos proporcionan una serie de tablas. Cada una de las tablas tiene una función concreta y tras estudiarlas se llegó a la conclusión de que estos cálculos son altamente complejos ya que no solo tienen en cuenta factores meteorológicos, sino que también tiene en cuenta la pendiente del terreno o el mes y la hora en el que se toma la muestra. Es por ello por lo que, dada la limitación de tiempo del periodo de prácticas, se tomó la decisión de abreviar de nuevo estos cálculos siempre y cuando se obtuviera un resultado lo más próximo al cálculo real. Le preguntamos de nuevo a la persona responsable que nos había proporcionado dichas tablas y nos comentó que podríamos utilizar para este prototipo y de manera orientativa y no precisa al cien por cien la tabla número 3 del documento para el cálculo de la probabilidad de ignición. Como se observa, se tienen en cuenta diferentes parámetros como es porcentaje de sombreado del cielo la temperatura en termómetro seco la humedad de combustible ligero muerto en porcentaje, la solución que se nos proporcionó fue sustituir este último parámetro por la humedad relativa del aire y que el resultado obtenido con la intersección de estos 3 parámetros se le sumase un 10% y que ese sería el resultado aproximado de la probabilidad de ignición. Es cierto que en un primer momento, no se quería aplicar este atajo debido a la ambición por querer representar unos resultados lo más realistas posibles, sin embargo,

había ciertos cálculos que hacían mucho más compleja esta práctica como el cálculo de la pendiente en cada uno de los puntos proporcionados, que a priori no suena excesivamente complicado pero indagando un poco sobre este tipo de toma de medidas, se observó que no todos los puntos podrían ser útiles para medir, que el cálculo de la pendiente no puede ser en un punto concreto sino que tiene que ser la interpolación entre un área seleccionada. Además, también se tenían en cuenta otros factores como la escala BU Ford para la fuerza del viento, la humedad del combustible muerto y algunos correctores extra que dependía de la orientación del viento, los meses o incluso las horas.

Sin embargo, una vez que se toma la decisión de utilizar la tabla descrita previamente, resulta que ningún modelo de Chat GPT es capaz de extraer la información a partir de esa tabla dado que no sabe leerla y al estar escaneada lo dificulta aún más. Es cierto que hay alguna librería de Python que precisamente es experta en extraer información de PDF escaneados, sin embargo, ya se añadiría un paso extra debido a que por un lado habría que extraer la información por ejemplo en texto plano de esta tabla escaneada y posteriormente pasárselo al modelo. Finalmente, se planteó una solución más directa que es la de pasar la información de la tabla a una base de datos en formato .db. De esta manera, la forma de calcular y la probabilidad de ignición es hacer una consulta SQL donde cada una de las columnas representa un parámetro, por lo que haciendo una query de SQL con la función where e introduciendo los parámetros de temperatura, humedad y estado del cielo, que extraiga a través del select la última columna que es la de la probabilidad de ignición. Por lo tanto, en esta herramienta el input es estos tres parámetros, la operativa es crear una consulta de SQL sobre una base de datos y devuelve el valor de la probabilidad de ignición en porcentaje.

En cuanto a la herramienta que calcula el índice de peligro de incendio, lo hace a partir de los parámetros probabilidad de ignición obtenida en la herramienta anterior y la velocidad del viento obtenida con la API del tiempo. A través de la tabla correspondiente que es más sencilla que la anterior, simplemente se ha generado un código en el cual hay diferentes bucles if donde dependiendo de la condición y del valor de la probabilidad de inicio y del viento se decreta una categoría de peligro de incendio que puede ser: sin

riesgo, prealerta, alerta, alarma y alarma extrema. Por lo tanto, en esta herramienta, el input es la probabilidad de ignición y la velocidad del viento en kilómetros por hora, el proceso a través de bucles condicionales y la salida es alguna de estas categorías mencionadas anteriormente.

6. Generación de mapas de calor e interpolación espacial

Jaquenoud, T. (2021). *AI Geospatial Wildfire Risk Prediction*. Kaggle. <https://www.kaggle.com/code/thjaquenoud/ai-geospatial-wildfire-risk-prediction>

En este apartado se explicará y analizará la herramienta de generación de mapas de calor que constituye la fase final del proceso de análisis espacial y geográfico. En este módulo confluyen y se materializan como resultados los valores numéricos obtenidos en etapas previas. A esta herramienta se le puede solicitar la generación de distintos mapas dependiendo del parámetro escogido, es decir, la generación del mapa puede ser en función de la probabilidad de ignición o de alerta del riesgo de peligro, casos en los que el camino para obtener estos valores ha sido altamente complejo o por otro lado, el mapa puede representar el parámetro del viento, temperatura o humedad, lo que permite que no solo tengamos un solo mapa sino que el usuario tenga la posibilidad de escoger qué tipo de visualización le puede ayudar más ya que por ejemplo, en el caso de que en una zona geográfica concreta haya una probabilidad de ignición mayor se puede ver qué motivo o parámetro de los mencionados anteriormente ha causado esa probabilidad elevada, obteniendo así la representación gráfica que ayude más a tomar una decisión.

El desarrollo de esta herramienta se inspira en el trabajo de Jaquenoud (2021), Cuyo proyecto de predicción geoespacial de riesgo de incendio mediante inteligencia artificial, empleando librerías de Python como *geopandas*, *matplotlib* y *scipy.interpolate*. Este estudio ha servido como referencia directa y ejemplo a seguir ya que es un trabajo sólido en el que básicamente a partir de puntos geográficos dispersos es capaz de obtener una representación con alto detalle gracias a la interpolación y técnicas de

densidad espacial. En el caso de este trabajo fin de máster, la herramienta `generate_heatmap` tiene el objetivo de transformar valores de diferentes puntos geográficos concretos de una malla en un mapa de calor interpolado que refleje según su color el grado de intensidad sobre el territorio para cada uno de estos parámetros. Por lo tanto, la información de entrada de esta herramienta es el parámetro que se desea representar, las coordenadas de cada uno de los puntos y el valor de dicho parámetro seleccionado para cada uno de estos puntos. A partir de esta entrada, se construye un *GeoDataFrame* mediante la librería *geopandas*, convirtiendo los pares de coordenadas en geometrías del tipo *Point* y proyectándolos al sistema métrico EPSG:3857, que se usa comúnmente en este tipo de representaciones como es en el caso del trabajo citado al comienzo de este subapartado. Una vez que se ha obtenido todos estos valores de manera satisfactoria, se procede a la interpretación espacial para la que se utiliza la función `griddata` del paquete *scipy.interpolate*, la cual genera una superficie continua a partir de un conjunto de puntos. Esta técnica también es comúnmente conocida en este tipo de estudios ambientales geofísicos y que nos asegura obtener unos resultados visualmente coherentes y útiles. Al igual que en el trabajo de Jaquenoud (2021), se optó por el método de interpolación cúbica, que básicamente ofrece una visualización más suave donde no hay cambio brusco o discontinuidades.

El mapa se representa mediante la función `pcolormesh` de *matplotlib*, utilizando una paleta de colores en la que el verde se refiere a valores bajos de peligro o bajos de temperatura o cualquier otro tipo de parámetro y el rojo para valores más altos y púrpura como valores extremos altos. Además, en el caso de que el parámetro escogido sea “índice de peligro”, dado que no es un parámetro numérico y son categorías cualitativas previamente definidas (*Sin riesgo, Prealerta, Alerta, Alarma, Alarma extrema*), se añade una leyenda mostrando qué color corresponde para cada uno de estos casos.

Una característica relevante de la herramienta y que supone un salto de calidad en la visualización y representación final se trata de la integración con *contextily*, que permite superponer el resultado interpolado sobre un mapa base de tipo satelital (Esri World Imagery). Esto hace que esta capa de

color que representa cada uno de los parámetros quede superpuesta sobre el territorio real mostrando tanto zonas forestales como núcleos urbanos. De esta manera los resultados adquieren más información aún ya que no solo se puede identificar donde se concentra el riesgo, sino que también la infraestructura o accidente geográfico en las zonas potencialmente afectadas.

En conjunto, la herramienta de generación de mapas de calor es quizás una de las más complejas a nivel de programación ya que combina rigor técnico y eficiencia computacional. al estar basado en librería estándar de Python, garantiza su adaptabilidad tanto a otros parámetros como a otros escenarios geográficos mientras que solo hay que variar la información en la entrada. Es por ello por lo que es el módulo encargado de dar evidencias y de construir un escenario eficiente para transmitir toda la información de una manera didáctica y visual para el usuario.

7. Visualización en la interfaz de Streamlit

Talaviya, A. (2022, diciembre 28). *A Complete Guide to Building Streamlit Data Web Apps with Code Examples*. Medium. <https://medium.com/@avikumart /a-complete-guide-to-building-streamlit-data-web-apps-with-code-examples-6cc1e42b2397>

A continuación, se presenta el último su apartado del marco teórico. Este módulo permite la visualización de los resultados en una interfaz gráfica, es decir, una vez que se han obtenido todo tipo de datos, desde puntos geográficos, generación de mallas parámetro meteorológico, cálculos avanzados, generación de mapas de calor, la etapa final es esta y radica en ofrecer al usuario un entorno que permita interactuar con el sistema de forma intuitiva y eficiente. Se han evaluado diferentes alternativas para interfaces en Python como Flask, Dash y Panel, sin embargo, se determinó que Streamlit era la opción más adecuada debido a su simplicidad técnica y su adaptabilidad con librerías de análisis y visualización de datos de Python.

El artículo de Talaviya (2022) ha sido un referente y resultó especialmente útil durante la creación de este módulo debido a que presenta un ejemplo práctico de cómo construir una aplicación web de datos en Streamlit desde cero utilizando un nivel técnico no muy avanzado y aplicable a distintos casos de uso. El tutorial citado explica de manera clara y entendible cómo configurar la estructura de una página, incorporar widgets, cargar datos y actualizar visualizaciones dinámicamente. Todo esto hizo que se seleccionara Streamlit como interfaz ya que la parte principal y más importante de este proyecto es en realidad la orquestación de todas las herramientas anteriores, este último módulo simplemente es una implementación interactiva a la hora de realizar la demostración, por lo tanto, no se trata de una necesidad imperiosa de realizar una interfaz de alto nivel técnico con un front de nivel profesional.

La herramienta descrita actúa como punto de unión entre el usuario y el agente orquestador. Como se muestra en una imagen a continuación o en la propia demo de este trabajo, se compone de un título, una barra lateral de control y un área central donde se muestran los mapas generados. Desde la barra lateral, el usuario introduce la ubicación del centro geográfico deseado, el tamaño del área de la malla que se estudiara, el espaciado entre puntos dentro de la malla y por último el tipo de mapa de calor que desea visualizar, es decir, se escoge el parámetro entre viento temperatura humedad, probabilidad de ignición o riesgo de peligro. Estos parámetros se capturan mediante *widgets* de entrada (`st.text_input`, `st.slider`, `st.selectbox`) que permiten validar valores y mantener una experiencia de usuario uniforme. El sistema ejecuta entonces una serie de funciones integradas y básicamente todas las que hemos mencionado anteriormente: `get_location`, `get_nearby_locations`, `get_weather`, `get_fire_risk`, `compute_fire_danger_from_points` y `generate_heatmap` controladas por el estado de sesión de Streamlit. Gracias a la persistencia de estado (`st.session_state`), los resultados obtenidos se almacenan temporalmente en memoria, lo que evita regenerar los mismos mapas más de una vez reduciendo el gasto de consumo extra, mecanismo inspirado en la buena práctica descrita por el artículo mencionado anteriormente la cita.

A continuación, un ejemplo de la apariencia de la interfaz de Streamlit:

The screenshot shows a Streamlit web application titled "Heatmaps de Riesgo de Incendios". On the left, there is a sidebar with the heading "Parámetros de entrada". It contains the following controls: a text input for "Lugar / topónimo" with the value "La Palma, Islas Canarias, Esp"; a slider for "Tamaño del cuadrado (lado en km)" with a value of 48; a slider for "Espaciado entre puntos (km)" with a value of 8; a dropdown menu for "Tipo de heatmap" with the selected option "indice_peligro"; and a "Generar mapas" button. The main area of the application has a heading "Heatmaps de Riesgo de Incendios" and a subtext "Selecciona ubicación, área y espaciado. Los mapas se generarán una sola vez y luego podrás cambiar entre ellos sin esperas." Below this, a yellow message box states "Aún no hay mapas generados. Pulsa 'Generar mapas' para iniciar el proceso."

Ilustración 1. Interfaz de Streamlit

Es cierto, que la demostración de este proyecto se hace a través de la aplicación de este Streamlit, no obstante, la capacidad de este sistema no radica en lo que se muestra en este interfaz. Esto se debe a que al propio modelo se le puede mandar una solicitud en lenguaje natural por parte del usuario y no necesariamente el resultado tiene que ser generar un mapa, le puedes preguntar sobre cualquier cosa que pueda hacer con las herramientas que tiene a su alcance por lo que no es un simple código al que le das ejecutar con unos parámetros de entrada específicos y te genera un conjunto de mapas, no es solo eso, además le puedes preguntar cuál es la humedad en una zona específica o que te haga una tabla completa con las coordenadas de 20 puntos diferentes de la provincia de Madrid con los valores de temperatura viento, humedad y probabilidad de ignición por poner un ejemplo concreto. Incluso puedes preguntarle sobre cálculos sencillos como puede ser la media de la temperatura en la provincia de Madrid. En este caso el agente automáticamente generará una malla con los puntos que estimen necesarios teniendo como centro geográfico Madrid, posteriormente calculará la temperatura en cada uno de esos puntos y posteriormente el propio modelo hará una media como es este caso o lo que se le solicite. Con estos ejemplos se pretende esclarecer que el hecho de haber realizado una interfaz dinámica para el usuario no significa que la capacidad del agente se limite únicamente a la generación de mapas, sino que a través de una

solicitud por parte del usuario es capaz de utilizar las herramientas que tiene a su alcance de la manera que éste desee.

En síntesis, el desarrollo de la interfaz con Streamlit ha permitido completar con una capa de interacción clara y operativa al sistema de este trabajo fin de máster. Gracias a la inspiración y a la referencia en el ejemplo de Talaviya (2022), la aplicación implementada demuestra soluciones de visualización en Python que mejora de manera considerable la interacción de los resultados con el usuario final logrando una comprensión visual esencial para la toma de decisiones teniendo este agente de emergencias como ayudante técnico.

El marco teórico desarrollado muestra las bases conceptuales y tecnológicas a partir de las cuales se construye el sistema de agentes inteligentes propuesto. Gracias a cada uno de los distintos subapartados, se ha ido justificando la elección de cada componente, de cada técnica y de cada función y siempre con una referencia técnica citada de carácter profesional que han servido de referencia y de orientación para el desarrollo de cada uno de los módulos. Desde el agente orquestador y el agente ReAct, pasando por cada una de las herramientas, tanto de obtención de datos como de cálculos avanzados, terminando con las herramientas de generación de mapas y el desarrollo de la interfaz.

En conjunto, esta combinación demuestra como la inteligencia artificial generativa es capaz de integrar distintas tecnologías para que gracias a un agente orquestador puedan converger y ser un sistema completo coherente y funcional. La arquitectura de un agente ReAct como la base del razonamiento cognitivo teniendo a su alcance diferentes herramientas, junto a la experiencia profesional de los técnicos de la empresa XRF.ai, hacen de este marco teórico una base sólida para el desarrollo metodológico posterior, teniendo asegurado fundamento científico tecnológico y operativo consistente para su posterior aplicación práctica.

Capítulo 4: Marco Metodológico

El presente apartado describe la metodología seguida para el desarrollo e implementación del agente de emergencias, elaborado como una aplicación basada en inteligencia artificial generativa y orientada a la predicción del riesgo de incendios forestales. Después del capítulo anterior donde se ha tratado en marco teórico y se expusieron los fundamentos conceptuales y tecnológicos que sustentan el proyecto, ahora, este apartado aborda el procedimiento metodológico. El trabajo consta de una investigación aplicada de tipo descriptivo experimental orientada a la validación funcional de un sistema novedoso y tecnológico como es los sistemas multi agentes de la IA generativa. Esta metodología combina elementos y técnicas de ingeniería de software, análisis de datos ambientales, diseño de un agente orquestador e integración de herramientas tanto de visualización y de cálculos avanzados como de obtención de información.

El capítulo se organiza en cinco subapartados. En primer lugar, se define la población y muestra de estudio en la que se basa este trabajo para obtener la muestra que representa dicha población. En segundo lugar, las variables intervinientes e instrumentos de evaluación, donde se detallan las variables que intervienen a la hora de la puesta en práctica de este sistema y los instrumentos que se usan para la posterior evaluación de resultados. A continuación, se detalla el diseño experimental que describe el método de investigación elegido y su procedimiento. Por último, el modelo de análisis de datos, donde se expondrá tanto un análisis cualitativo de las salidas generadas del sistema complementado con observaciones cuantitativas básicas que nos aportarán información a nivel de rendimiento y efectividad.

1. Población y muestra

La población objeto de estudio de este proyecto está formada por el conjunto de zonas geográficas que pueden ser analizadas por el agente de emergencias, dado que el sistema desarrollado permite procesar por tanto información

meteorológica como geoespacial en cualquier punto del territorio, la limitación para estudiar cualquier población es prácticamente nula. Como se ha comentado en apartados anteriores, el único impedimento en este aspecto es que, a la hora de introducir el nombre del centro geográfico de interés, debe ser detallado y específico. En el apartado de Marco Teórico ya se comentó que podía haber confusión si solo se le introduce la palabra Córdoba como centro geográfico ya que en este caso existe una ciudad tanto en Argentina como en España y que habría que referirse de una manera más específica como, por ejemplo: Córdoba, ciudad de la Comunidad Autónoma de Andalucía en el sur de España.

Sin embargo, en el ámbito operativo se ha centrado la validación del sistema en un punto geográfico concreto. Esta zona geográfica en concreto se trata de la isla de La Palma en el archipiélago canario, España. Esto se debe a que como se ha comentado en varias ocasiones en este trabajo, la elaboración técnica de este sistema se ha forjado en las prácticas externas del máster de inteligencia artificial en la empresa XRF.ai. Resulta que dicha empresa mantiene una colaboración directa con las unidades de emergencias de esta ubicación. Es por ello por lo que la elección de La Palma como población de referencia se debe a que, puestos a elegir cualquier lugar geográfico, se elige este que es básicamente donde en un futuro será la práctica habitual para las personas responsables de la toma de decisiones en las emergencias de la isla de La Palma.

En cuanto a la muestra, esta se define a partir de un conjunto de variables donde se puede elegir el tamaño de la zona a representar y cuántos puntos geográficos generados dentro de esta área que permita evaluar la zona de manera precisa y coherente en condiciones reales de aplicación. Como se observa, esta arquitectura tiene la cualidad de ser altamente versátil y flexible por lo que cada uno de estos puntos tratados en este apartado pueden ser perfectamente escogidos a gusto del usuario.

2. Variables intervinientes e instrumentos de evaluación

El desarrollo metodológico del agente de emergencia se fundamenta de la medición de un conjunto de variables que permiten evaluar tanto las condiciones ambientales como el rendimiento funciona del propio sistema.

En cuanto a la variable meteorológica, representan la base de los cálculos y son la temperatura, la humedad relativa del aire, la velocidad del viento y el estado del cielo. Estos parámetros son obtenidos en tiempo real a través de la API de Open-Meteo, proceso que ya se describió detalladamente en el apartado de Marco teórico dentro del subapartado de API meteorológica para la obtención de datos en tiempo real. Al obtener estos datos actualizados para cada una de las coordenadas geográficas específicas que se estudia obtenemos a partir de más herramientas más resultados como la probabilidad de ignición o el estado de alerta en cuanto al índice de peligro de incendio estimado. Por otro lado, las variables geoespaciales a través del módulo de geocodificación Nominatim, también explicado de manera detallada en el capítulo de Marco teórico dentro del apartado de proceso de codificación y obtención de coordenadas y por otra parte el generador de mallas geográficas detallado en el apartado de generación de mallas geográficas y muestreos espaciales. Todos estos puntos actúan como unidades de observación y permite al agente reunir toda la información necesaria tanto en el aspecto geoespacial como meteorológico para poder generar unos mapas de calor como resultado lo más precisos posibles.

Además, se utilizan variables asociadas al rendimiento técnico del agente como son el tiempo de ejecución y el consumo de tokens en el modelo de lenguaje, datos estrechamente relacionados con el área procesada y la densidad de puntos que se estudian dentro de esta área. Por otro lado, se utiliza unos instrumentos de evaluación en cuanto a la coherencia visual y semántica de los mapas generados analizando la quinta ciudad de estos para transmitir toda la información calculada y obtenida a partir de las herramientas a lo largo del proceso operativo. En conjunto, el uso combinado de esta variable y herramientas asegura una evaluación fiable del sistema

abarcando tanto la viabilidad de los datos procesados como la calidad visual y representativa de los resultados obtenidos.

3. Diseño experimental

El Diseño experimental adoptado en este proyecto tiene la finalidad de analizar la viabilidad funcional de un sistema con un agente orquestador inteligente basado en inteligencia artificial generativa capaz de integrar diferentes herramientas y ejecutarlas para obtener resultados en tiempo real. este trabajo por lo tanto no busca un enfoque metodológico únicamente para contrastar una hipótesis, sino que su misión es demostrar la operatividad precisión y adaptabilidad a través de una demo para la posibilidad de que se pueda usar en un futuro como un asistente real de emergencia por riesgo de incendio.

El procedimiento seguido se basa en la estructura detallada en el bloque anterior del Marco teórico. En primer lugar, se estudió qué tipo de agente orquestador es el idóneo para el poder solventar los retos y exigencias de este proceso. En este caso se selecciona al agente ReAct como núcleo de razonamiento lo que implica una configuración dentro del framework de Langchain. A continuación, se construyen un abanico de herramientas que son a las que tendrá acceso este agente orquestador y las cuales se ejecutarán en el momento y orden que decida el agente según la solicitud el lenguaje natural por parte del usuario. Una vez que ya están integradas todas estas herramientas dentro del agente, se procede al desarrollo de la aplicación de Streamlit, técnica que permite generar una interfaz amigable haciendo de nexo de comunicación entre el agente y el usuario. Finalmente, y después de un proceso de validación en el que se han realizado diferentes pruebas para afinar algunos detalles concretos, se obtiene la demo que se estaba buscando a partir de la cual podemos obtener unos resultados que se estudiarán en los siguientes apartados de este trabajo fin de Máster.

4. Modelo de análisis de datos

El modelo de análisis de datos empleado en este proyecto tiene un enfoque mixto ya que combina tanto técnicas de evaluación cuantitativas como cualitativas debido a la naturaleza híbrida que compone el sistema de este agente de emergencias. La función de este sistema no radica solamente en procesar datos meteorológicos o geográficos a través de diferentes herramientas, sino que además integra un razonamiento autónomo que implica la planificación de la ejecución de cada una de estas herramientas y unos resultados plasmados en representaciones visuales que deben demostrar de manera precisa y entendible toda la información de los datos de salida generados. Por esta razón, el análisis de datos se orienta en estas dos ramas: por un lado, la evaluación cuantitativa del rendimiento técnico del agente y por otro lado la evaluación cualitativa en cuanto a la coherencia y precisión visual de los resultados obtenidos. Este modelo mixto permite analizar la arquitectura creada desde diferentes puntos de vista ya que se cree que una validación eficaz no debe depender exclusivamente de indicadores numéricos, sino que también de la interpretación del producto final que se le entrega al usuario. En este tipo de arquitecturas de error de cálculo no solo puede venir debido a una mala programación a la hora de calcular un valor o dato concreto, sino que también puede venir de que el agente orquestador no ha entendido de manera correcta la pregunta o no ha sabido interpretarla, lo que deriva en que la elección de la ruta a seguir a través de los distintos módulos no ha sido la correcta. De esta manera el proceso de análisis no debe limitarse a medir tiempos o consumos solamente, sino que también a cómo el agente utiliza sus recursos para llegar a un resultado coherente y útil.

Comenzando con la evaluación del análisis cuantitativo del rendimiento del sistema, a lo que se refiere a la capacidad del agente para ejecutar la tarea proporcionada por parte del usuario de manera precisa y eficaz. Por lo tanto, en este análisis se implementaron un conjunto de métricas que permiten obtener una visión objetiva de su funcionamiento.

Entre las métricas más relevante se encuentra el tiempo de ejecución. Este tiempo es el comprendido entre la solicitud inicial del usuario y la generación del resultado final, es decir, desde que el usuario introduce su solicitud como input y se le da ejecutar hasta que en la aplicación de streamlit se muestren de manera definitiva cada uno de los mapas de calor para cada parámetro. Esta medida resulta imprescindible para poder valorar la eficiencia del sistema y más si tenemos en cuenta el contexto de que este va a ser una herramienta para la obtención de datos en tiempo real que ayude al encargado de las emergencias en cuestión de una toma de decisiones, por lo que el tiempo apremia en este tipo de situaciones. El tiempo de ejecución se ha monitorizado de manera directa desde el entorno de Python. Sin embargo, hay que tener en cuenta algunos factores que hacen que este tiempo varíe. Para empezar, se debe tener en cuenta que vamos a estudiar dos tiempos de ejecución, el primero de ellos es cuando no se ha inicializado la aplicación de Streamlit y hay que lanzar todos los procesos de nuevas, coloquialmente hablando. Por otro lado, la otra medida de tiempo de ejecución se basa en el tiempo una vez que ya se han obtenido ciertos resultados en la aplicación de Streamlit y que el usuario modifica diferentes parámetros como el área a tener en cuenta de los mapas o el espaciado entre puntos o incluso el centro geográfico de estos mapas. Como se ha comentado en apartados anteriores, se guarda cierta información una vez que se ha realizado la primera ejecución para que la siguiente ejecución ya con la aplicación iniciada sea más reducidas en cuanto al tiempo destinado.

Otra variable cuantitativa relevante es el consumo de tokens. Al final estamos utilizando en este caso el modelo GPT4-omini a través de Azure OpenAI, lo que conlleva un consumo de tokens según los recursos utilizados. Dependiendo del modelo seleccionado los tokens son más o menos caros. Por lo tanto, los resultados obtenidos se obtienen a partir de la monitorización del consumo de tokens por interacción de la página de Azure conectada a través de una conexión API con este sistema. No obstante, en esta variable también intervienen diferentes factores. Por un lado, intervienen el tamaño de la malla y la densidad de sus puntos ya que un mayor tamaño y mayor densidad de puntos conlleva una mayor obtención de datos y de cálculos, lo que radica, como es lógico, en un mayor consumo de tokens. Por otra parte, también

depende el modelo que se utilice, en este caso hemos utilizado GPT4-omini ya que es un modelo útil, barato y con capacidad de obtener resultados precisos. El hecho de que sea un modelo barato dentro del catálogo a elegir implica que tenga una relación inversa con el tiempo ya que cuanto mejor sea este modelo, más rápido y mejor se dará la ejecución, pero más caros serán los tokens utilizados. En este caso, como hemos mencionado de manera reiterada en este documento, el agente es un prototipo y el objetivo no es obtener la mejor respuesta posible en el menor tiempo posible con un presupuesto dado, sino que es tratar de analizar la capacidad operativa de un sistema con agentes de estas características y es por ello por lo que se ha utilizado este modelo cuyos resultados son coherentes y cuyo coste es mínimo. Por lo tanto, el primer requisito para todas estas configuraciones es que el sistema responda de manera correcta y precisa a la solicitud del usuario. Una vez logrado ese hito, se modifica estos parámetros para que el consumo sea mínimo, ya que nos encontramos en un proyecto de investigación donde hay cantidad infinita de pruebas y validaciones teniendo en cuenta que el consumo de tiempo no sea extremadamente elevado y sea de como máximo 2 minutos. En caso de elevar el nivel de este proyecto a un carácter profesional, implicaría ajustar dichos parámetros de manera que ellos tengan a partir de ello los mejores resultados dentro de un presupuesto, sin embargo, como no es el caso y los tokens consumidos se reflejan directamente en una cuenta de estudiante con unos créditos limitados disponibles. Por lo tanto, el conjunto de las métricas cuantitativas permite evaluar de manera objetiva la eficiencia del sistema a través de un equilibrio entre precisión y velocidad y, además, nos aporta una idea de los recursos necesarios para poder implementar este agente en un entorno profesional.

En cuanto al análisis cualitativo, éste se refiere a la evaluación del comportamiento del sistema de una perspectiva centrada en la precisión visual y eficiencia práctica de los resultados generados. Por lo tanto, se analiza la calidad de las representaciones visuales de los mapas de calor generados ya que de nada sirve obtener unos resultados que son incapaces de ser transmitidos y ser interpretados por el usuario final. Para ello a partir del tutor de prácticas, que es el jefe de departamento del área de inteligencia artificial de XRF, se evalúa la precisión visual y geoespacial haciendo énfasis

en la distribución cromática obtenida de valores tanto de temperatura, humedad, viento, probabilidad de ignición y el riesgo de incendio mediante el módulo de interpolación espacial. Se comprobó que las transiciones entre zonas de distinta intensidad de color se mimetizaban de manera progresiva y coherente guardando relación con la gama cromática representada en el margen de la figura. La técnica implementada de interpolación cúbica mediante la librería *scipy.interpolate*, ofreció una representación suave y continua permitiendo una visualización de una alta calidad. Además, se revisó detenidamente la correspondencia entre los resultados visuales y la realidad geográfica de la isla de La Palma, comprobando que cada una de las coordenadas descritas están representadas correctamente al igual que cada uno de los cálculos realizados cumpliendo con un carácter lógico. Esto último se refiere a que en caso de que el sureste de la isla este calificado con un riesgo de alarma mientras que el resto de la isla se valora con una alerta de riesgo menor, a través de la navegación con la aplicación de Streamlit por los distintos mapas, eres capaz de saber a qué se debe dicha alerta ya que desde mapa del riesgo de incendio puedes navegar hasta el mapa de probabilidad de ignición cuyos datos deben reflejar una mayor probabilidad en esta parte de la isla. En caso de que no, significa que el viento justo en esos puntos de la isla es considerablemente mayor al resto. Este caso es un ejemplo de supuesto lógico con los que se ha aprobado la viabilidad de los resultados ya que al fin y al cabo es esencial que todos los mapas guarden relación y que muestren el verdadero origen de un estado de alerta alto por ejemplo y saber si se debe al viento o a cualquier otro parámetro. Se trata de evaluar los resultados desde una perspectiva operativa para saber si este prototipo es capaz de observar inmediatamente el impacto en los resultados visuales.

Por último, se analizó la capacidad de este sistema variando los supuestos prácticos a partir de los parámetros seleccionables. Se probó con diferentes centros geográficos con diferentes tamaños para el área de estudio y se estudió la densidad de los puntos en distintas situaciones y contextos con el fin de averiguar las limitaciones a las que ponemos nos podemos enfrentar en este trabajo.

Gracias a todas estas evaluaciones se ha podido construir una idea general de las capacidades y limitaciones operativas de este sistema y que se mostrará en el documento dentro del siguiente capítulo: Presentación de análisis, resultados y discusión.

Capítulo 5: Presentación de análisis, resultados y discusión

Tras la descripción del marco metodológico, esta sección presenta los resultados obtenidos durante la fase de implementación y validación del proyecto. El propósito principal de este capítulo es exponer de manera clara y estructurada los resultados tanto numéricos como visuales, derivados del funcionamiento del sistema para su posterior análisis y finalmente su discusión. En primer lugar, se presenta el apartado de análisis descriptivo de la muestra donde se detallan de manera específica los diferentes datos y parámetros empleados. Siguiendo con el subapartado posterior, en el que se presentan los resultados obtenidos durante las pruebas funcionales del agente, mostrando su comportamiento ante diferentes situaciones y variaciones de parámetros para así obtener una vista lo más general y detallada posible de la operativa de esta arquitectura. Por último y tras el paso previo de obtención de resultados, se discutirá sobre la capacidad operativa de este sistema de agentes basado en los resultados previamente obtenidos. En conjunto este capítulo tiene como finalidad demostrar, a partir de la inteligencia artificial generativa, la capacidad operativa y el potencial del sistema diseñado en el contexto de la predicción de incendios forestales.

1. Análisis descriptivo de la muestra

La muestra seleccionada para el análisis y validación del agente de emergencias va a tener algunos parámetros fijos y otros variables. Comenzando por los fijos, usaremos la aplicación de Streamlit para la ejecución del sistema, el centro geográfico va a ser la isla de La Palma perteneciente al archipiélago canario en España. Esta decisión se debe a la

colaboración directa entre la empresa XRF.ai y las unidades de emergencia de dicha isla y, además, es la entidad que nos ha aportado la documentación necesaria pertinente para algunos cálculos en específico. Continuando con los parámetros fijos el siguiente va a ser el área de estudio en kilómetros cuadrados dado que al ser La Palma una isla, conviene que el estudio se realice sobre un área fija y abarque de manera lo más precisa posible el terreno de la propia isla para que no sea demasiado pequeña y deje zonas sin rastrear, ni sea demasiado grande y estemos tratando puntos fuera de la isla en pleno mar abierto. El último parámetro fijo que no va a variar son los resultados solicitados, es decir, siempre se va a pedir al agente los mapas de calor respectivos al viento, temperatura, humedad, probabilidad de ignición y riesgo de peligro de incendio. Esto sirve para estar centrado siempre en el mismo resultado ya que de otra manera se dispararía el número de variables y parámetros a tener en cuenta para cada casuística. Por otra parte, el parámetro que va a variar va a ser por un lado el parámetro de espaciado entre puntos, es decir, la densidad de puntos que se van a tomar como muestra de medida dentro del área seleccionada. Esto implica que se van a obtener coordenadas, datos meteorológicos y cálculos a partir de estos propios datos para cada uno de los puntos, lo que conlleva una relación directa con la velocidad de ejecución y los tokens consumidos. Se refiere a que el estudio se centra en la variación de este parámetro para comparar los resultados y observar las variaciones de estas medidas evaluativas y así comprender el funcionamiento y sus requisitos de tiempo y consumo.

Por otro lado, también se va a estudiar el tiempo y consumo que emplea en caso de que la ejecución se realice desde el momento cero y en caso de que la ejecución se realice una vez que la aplicación ya está iniciada y se vuelve a ejecutar con otros parámetros diferentes. Este planteamiento garantiza la validez del análisis facilitando la obtención de resultados comparables y representativos donde el agente de emergencias pueda demostrar su capacidad y operatividad en un entorno real de aplicación.

2. Resultados obtenidos

Reflejando el procedimiento descrito anteriormente se muestra a continuación las pruebas que se realizaron sobre la isla de la Palma manteniendo constante el área de análisis y variando la densidad de puntos de la malla geográfica. Se consideraron dos configuraciones principales: Por un lado, con un espaciado de puntos de 8 km entre cada uno de ellos y por otro con un espaciado de 3 km. La representación de ambas situaciones por parejas según el parámetro (temperatura, humedad relativa, velocidad del viento, probabilidad de ignición y nivel de alerta por peligro de incendio) para que se pueda observar cómo influye en el nivel de detalle visual y en la precisión local de los resultados.

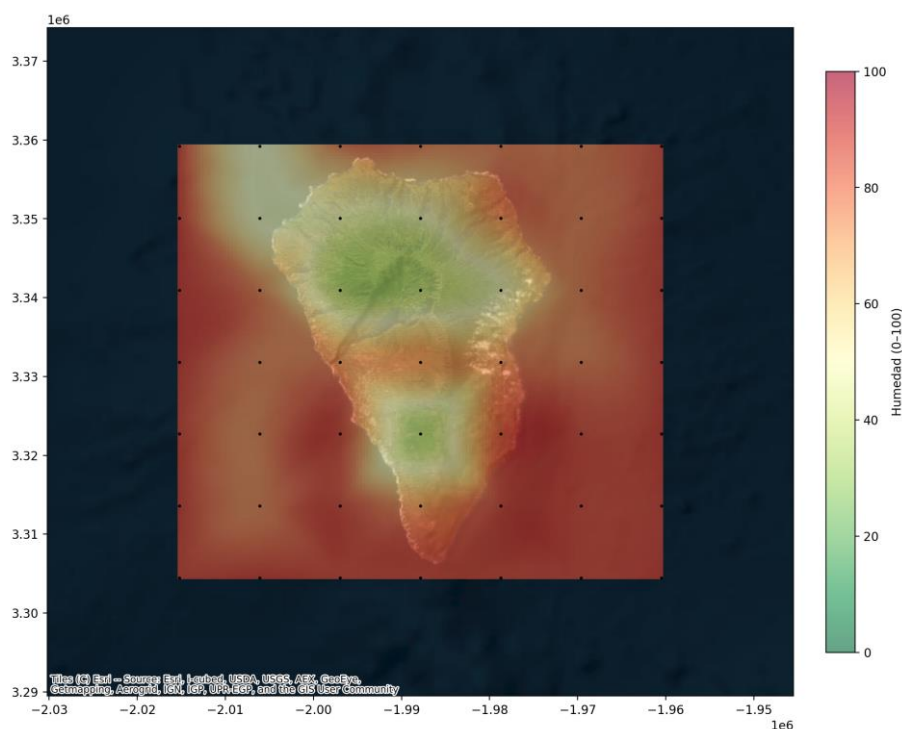


Ilustración 2. Mapa de Humedad con espaciado de 8 km

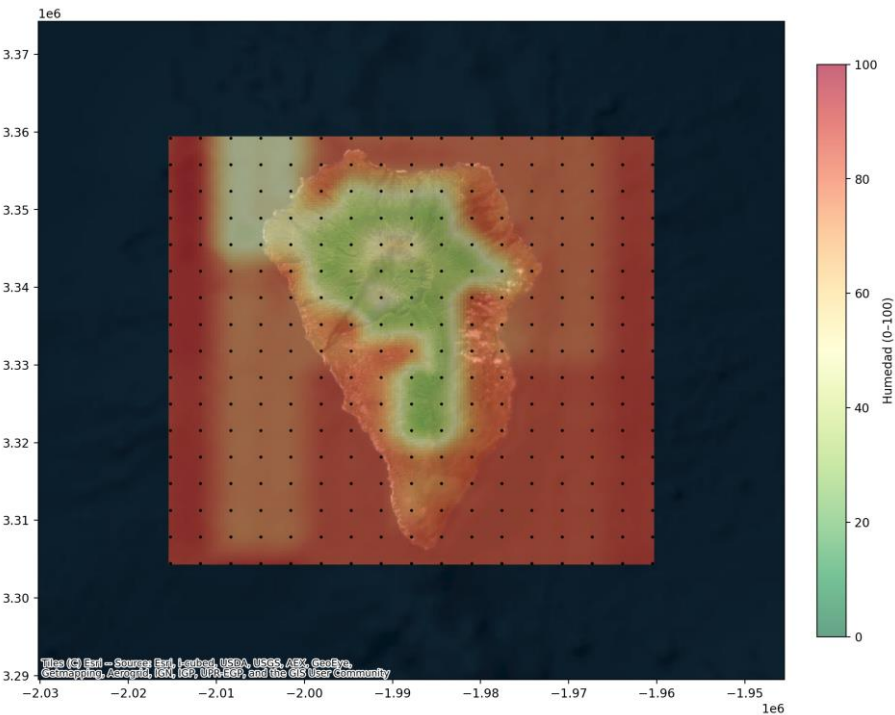


Ilustración 3. Mapa de Humedad con espaciado de 3 km

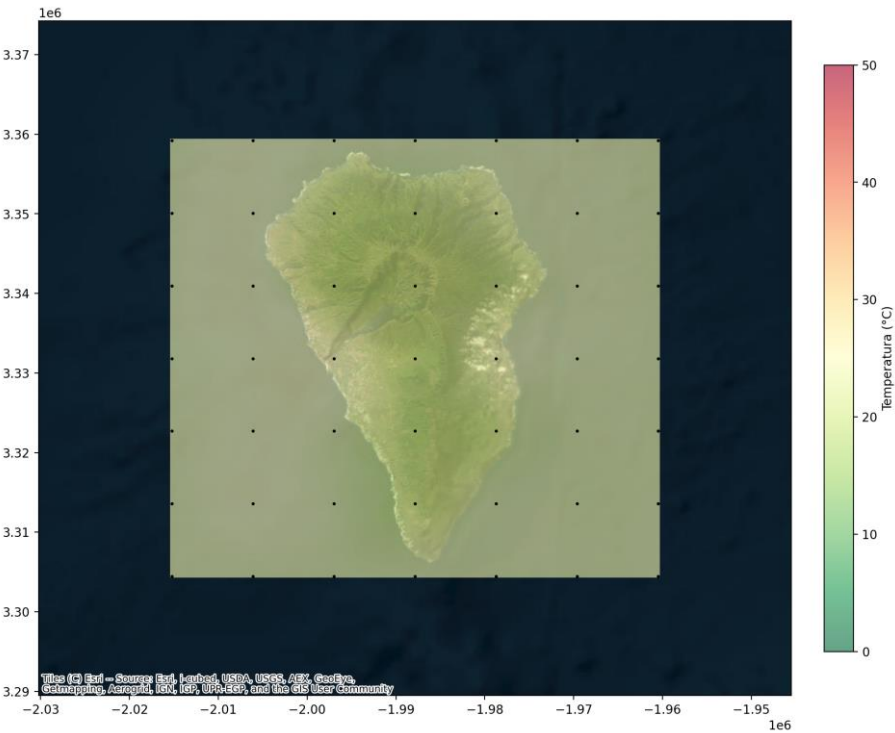


Ilustración 4. Mapa de temperatura con espaciado de 8 km

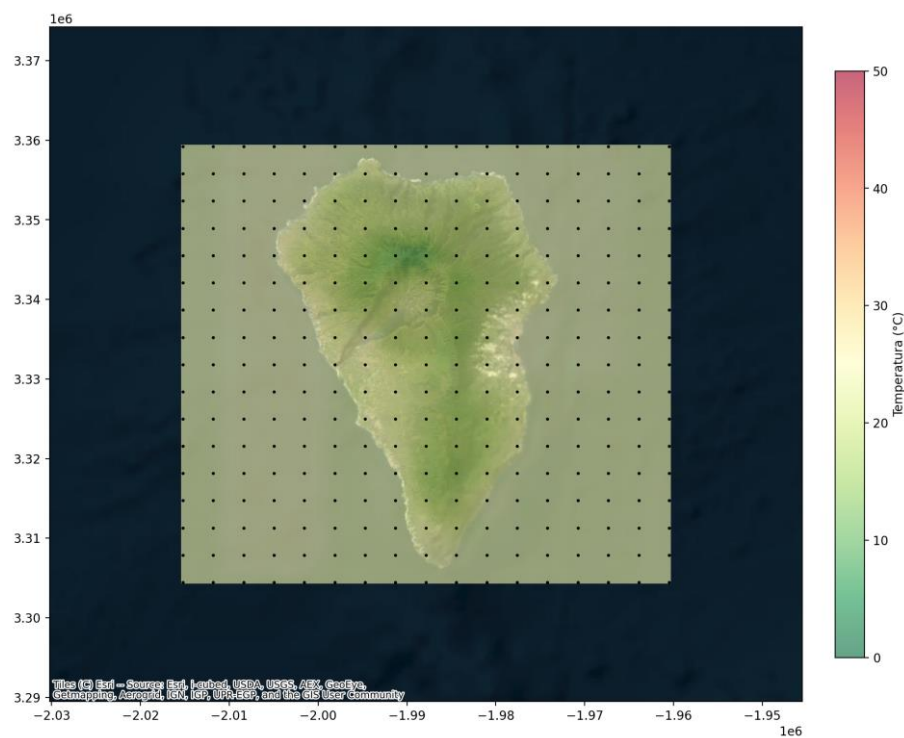


Ilustración 5. Mapa de temperatura con espaciado de 3 km

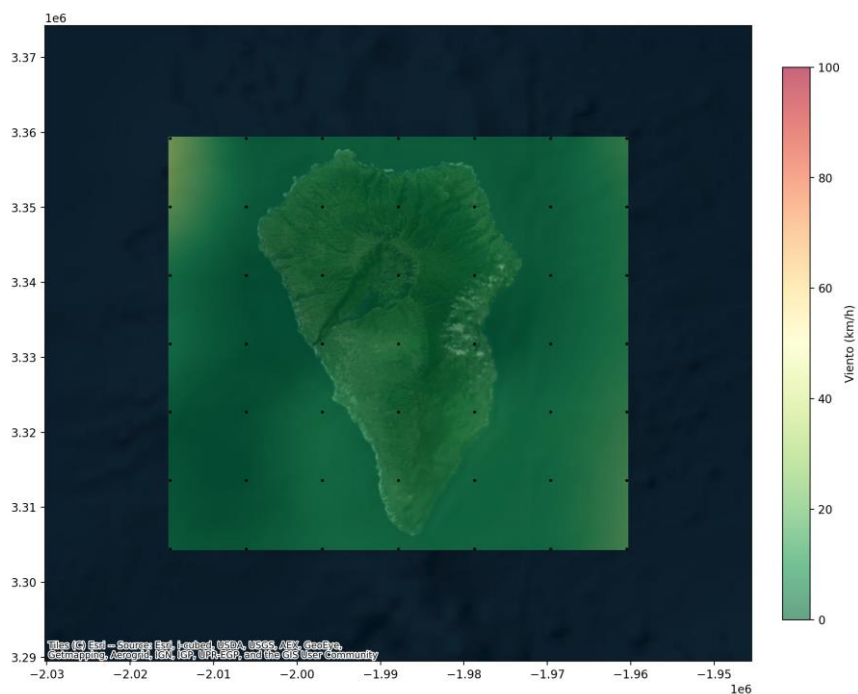


Ilustración 6. Mapa de viento con espaciado de 8 km

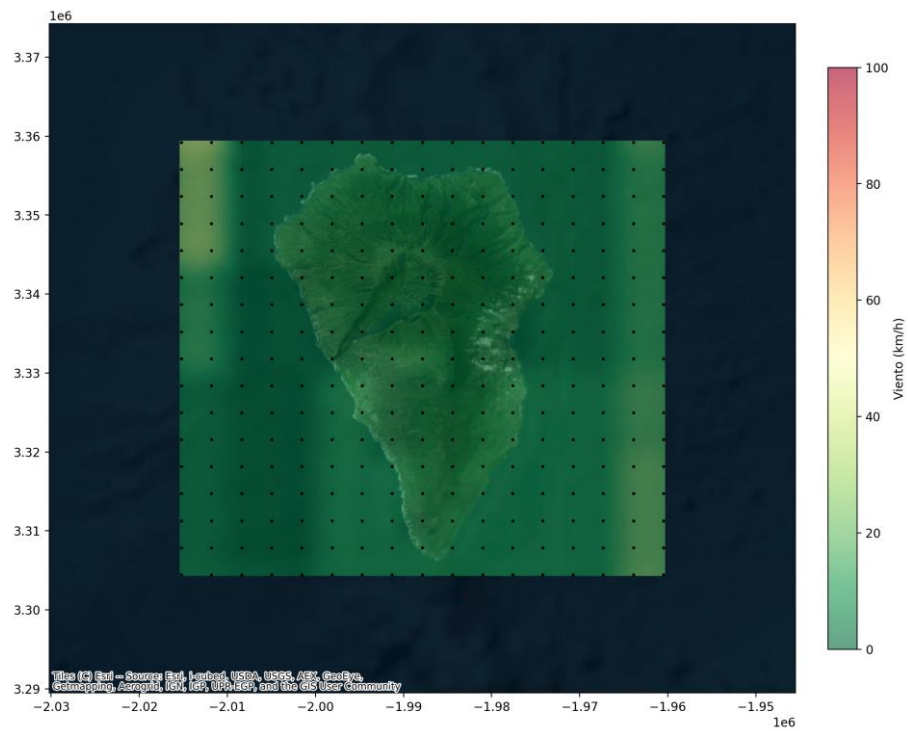


Ilustración 7. Mapa de viento con espaciado de 3 km

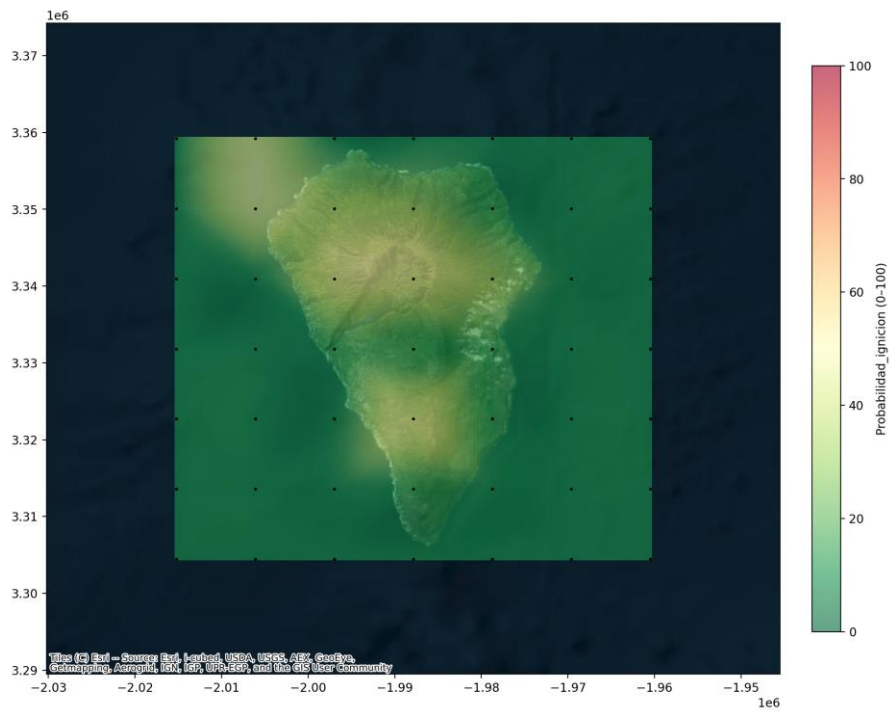


Ilustración 8. Mapa de probabilidad de ignición con espaciado de 8 km

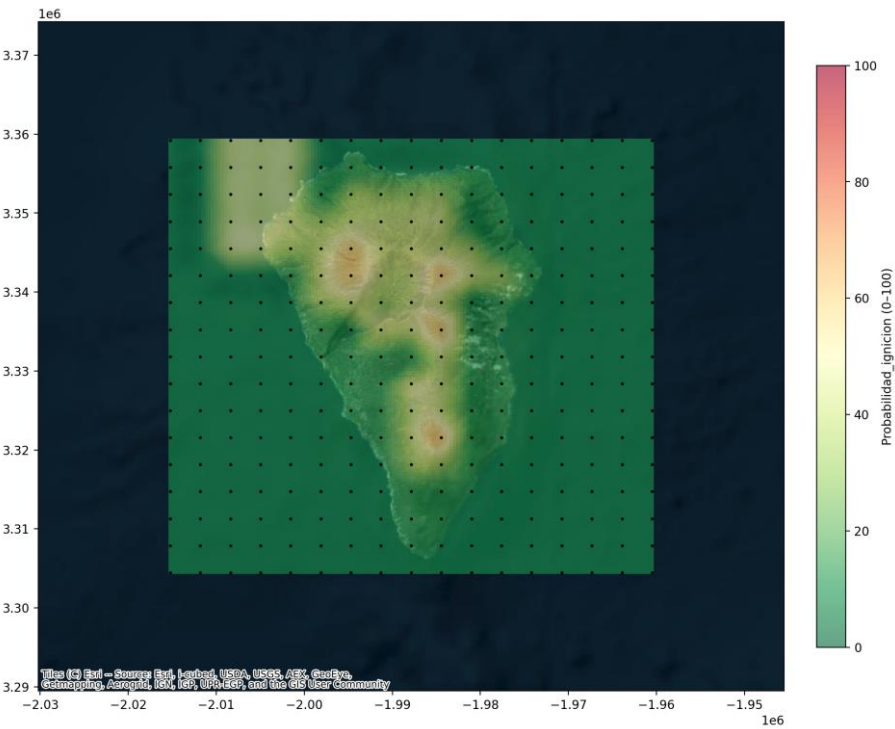


Ilustración 9. Mapa de probabilidad de ignición con espaciado de 3 km

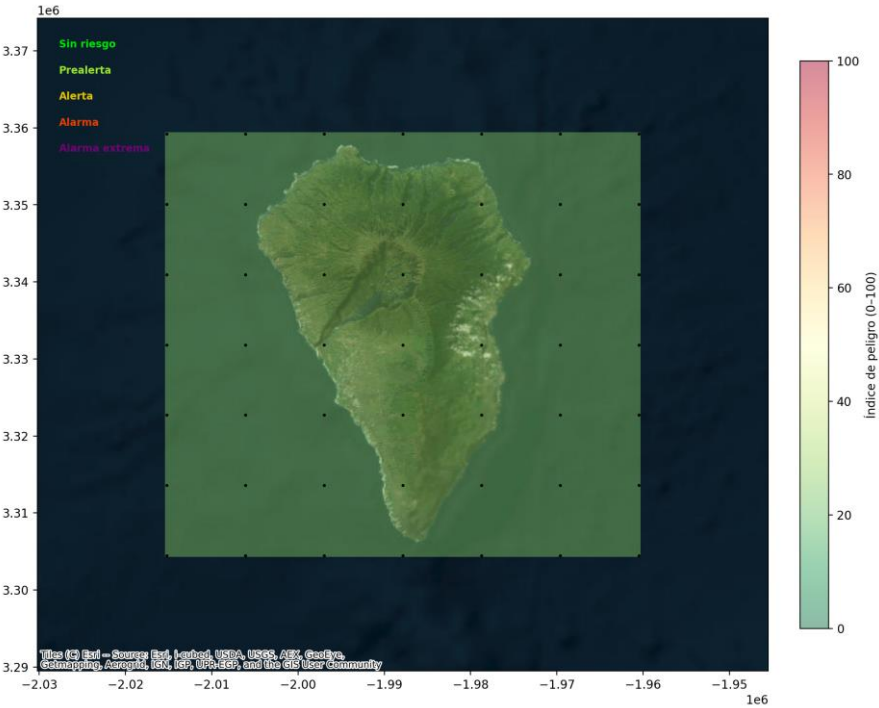


Ilustración 10. Mapa de índice de peligro con espaciado de 8 km

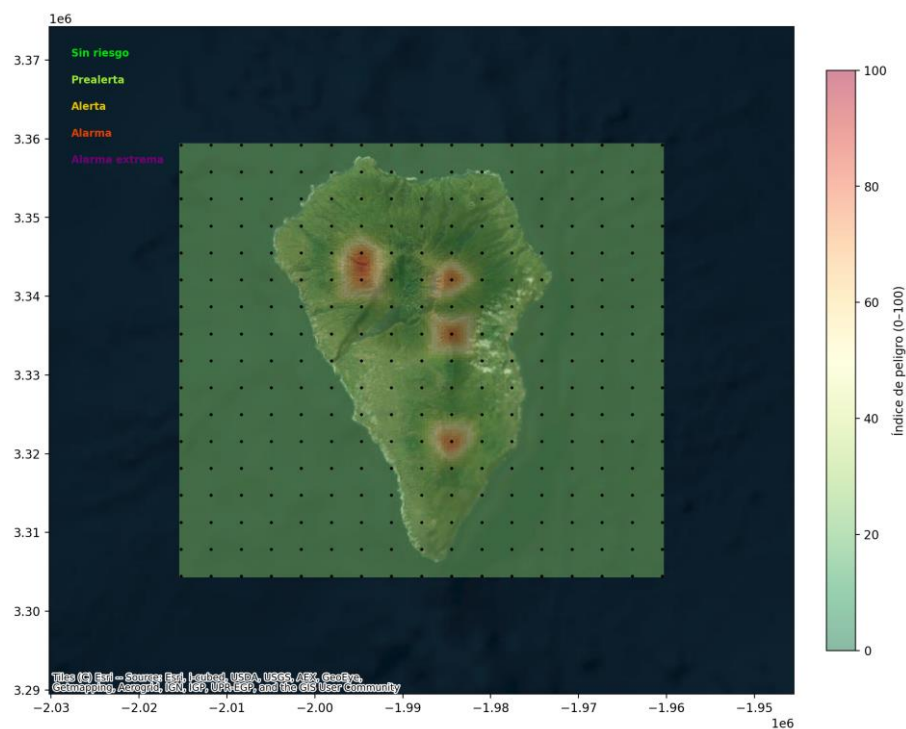


Ilustración 11. Mapa de índice de peligro con espaciado de 3 km

Además de las representaciones gráficas, se registraron métricas cuantitativas relacionadas con el rendimiento del sistema, que incluyen el tiempo de ejecución y el consumo de tokens durante las distintas ejecuciones. Las distintas pruebas se realizaron tanto con la aplicación iniciada como ya en funcionamiento y variando también el espaciado entre puntos de la misma manera y diferencia que en las imágenes representadas previamente.

Muestra	Espaciado (km)	Estado de la aplicación	Tiempo de ejecución	Consumo (tokens)	Coste (\$)
1	8	Iniciada	20 s	63.09 K	0.0150
2	8	Sin iniciar	36 s	66.26 K	0.0152
3	3	Iniciada	1 min 25 s	78.82 K	0.0159
4	3	Sin iniciar	1 min 40 s	79.55 K	0.0160

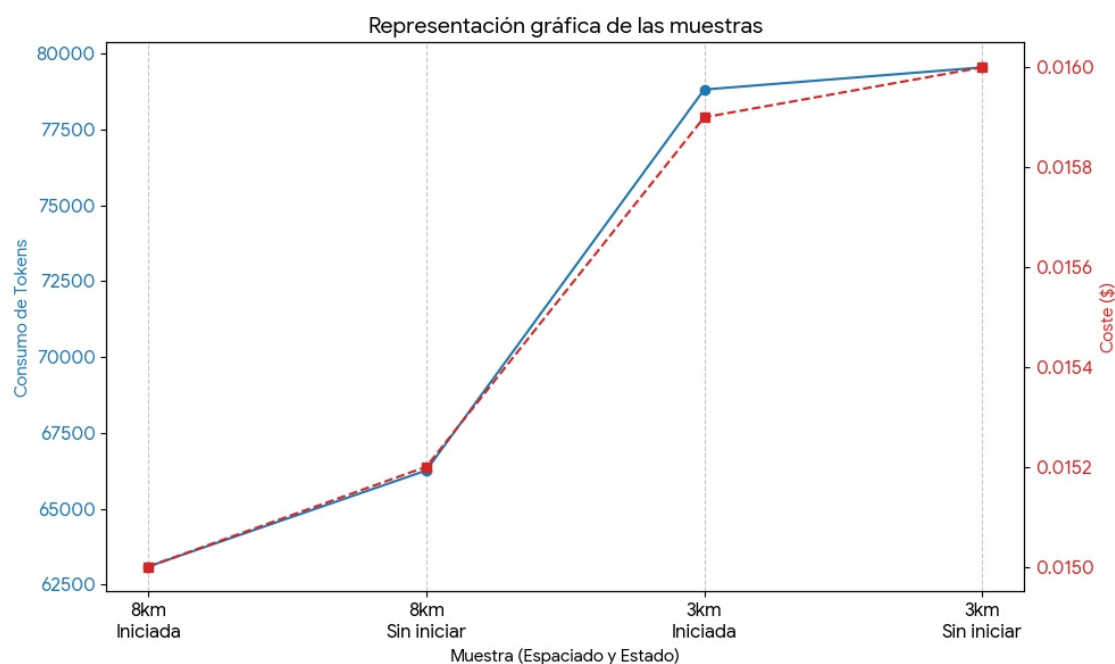


Ilustración 12. Representación gráfica de las muestras

El sistema mostró un tiempo medio de inicialización de ocho segundos, tras los cuales se procedió a la ejecución controlada mediante el panel de Azure y cronómetro local. Los resultados evidencian que una mayor densidad de puntos incrementa tanto el tiempo de cálculo como el consumo de recursos, mientras que la ejecución con la aplicación ya iniciada reduce de manera significativa la latencia.

3. Discusión y análisis de los resultados

Los resultados obtenidos durante la fase de experimentación de la siesta de agente de emergencias permiten constatar la capacidad técnica y funcional del modelo. La comparación de los mapas con distinta densidad de puntos evidencia que el aumento de resolución mejora la precisión local y la continuidad de las interpolaciones, aunque implica un incremento proporcional en el tiempo de cálculo y en el consumo de recursos. El análisis cuantitativo de tiempos de ejecución y consumo de tokens confirma la

eficiencia del sistema, especialmente en ejecuciones con la aplicación ya iniciada, donde el tiempo se reduce.

Desde el punto de vista práctico, este comportamiento respalda la elección del agente ReAct como núcleo operativo para tareas que requieren razonamiento iterativo y gestión de datos en tiempo real, mostrando un equilibrio entre rendimiento y calidad ofreciendo un soporte visual útil para la toma de decisiones en situaciones de emergencia. Estos resultados nos confirman que la arquitectura desarrollada cumple con los objetivos planteados en términos de operatividad, coherencia visual y eficiencia de ejecución, sentando una base sólida para las reflexiones finales que se desarrollarán en el capítulo siguiente de conclusiones.

Capítulo 6: Conclusiones

Volviendo al conjunto de objetivos específicos mencionados en el capítulo 2 de este trabajo, Objetivos del TFM, se pretende valorar el grado de cumplimiento. El primero de los objetivos específicos redactados se basa en estudiar los principios de la inteligencia artificial generativa dentro del ámbito de los sistemas multiagente con el fin de seleccionar la arquitectura y framework más adecuados para un agente orquestador dentro de este caso de uso. Dicho objetivo se ha alcanzado mediante un análisis exhaustivo con una familiarización general en este campo y tras la comparación de diferentes alternativas. Se estudió en profundidad tres tipos de agentes: ToolCallingAgent, CodeAgent y el agente ReAct llegando a la conclusión de que este último cumplía de mejor manera que el resto. Se integra con LangChain y LangGraph y su potencial reside en un bajo consumo y la capacidad de planificar y orquestar diferentes módulos a través del flujo Thought–Action–Observation. En consecuencia, el objetivo de fundamentar y elegir la arquitectura óptima puede considerarse cumplido.

El segundo objetivo específico reside en plantear la estructura funcional del sistema, definiendo tanto módulos como flujos de información que permitieran la interacción entre el agente y las herramientas a su alcance.

Para este punto se plantea una arquitectura con dos capas, la primera capa con herramientas de cálculo y extracción de datos y una segunda capa de visualización e interacción. Se definieron las dependencias funcionales de cada una de las herramientas que son: geocodificación, generación de malla, consulta meteorológica, cálculo de probabilidad de ignición, determinación del índice de peligro e interpolación para mapas de calor y la elección de Streamlit para la visualización. El objetivo de diseño queda, por tanto, satisfecho al haberse obtenido una arquitectura modular, flexible y coherente con los requisitos operativos.

El tercer objetivo específico se basa en implementar este agente orquestador con el framework seleccionado integrando de manera funcional cada uno de los módulos ya mencionados en el objetivo anterior. Por lo tanto, se construyó un agente orquestador de tipo ReAct capaz de interpretar la intención de usuario a partir de su mensaje de entrada y elegir que herramientas utilizar y en qué orden se ejecutan. El sistema integra la geocodificación sobre servicios abiertos, la consulta a la API meteorológica para temperatura, humedad y viento, el cálculo de probabilidad de ignición apoyado en tablas locales de referencia, así como la derivación del índice de peligro a partir de ignición y viento. Finalmente, la generación de mapas de calor interpolados a partir de la interfaz de Streamlit y además con la posibilidad de escoger el centro geográfico, el parámetro a visualizar y ajustar el área y densidad de muestreo.

En cuanto al cuarto objetivo específico, se orienta a la comprobación del funcionamiento operativos del sistema y evaluar sus respuestas para asegurar que sean eficientes y coherentes. Quizás este objetivo sea el que más complicación tiene y en el que más tiempo se ha invertido. Esto se debe a que se basa en que todos los diferentes elementos dentro de este proyecto funcionen de manera conjunta y dinámica sin pérdidas de información o redundancias. Para ello, se han modificado en numerosas ocasiones las herramientas e instrucciones del agente orquestador para poder lograr esta sintonía dejando sobre todo muy claro la breve explicación de la funcionalidad y elementos de entrada y salida de cada una de las herramientas en el System Prompt del agente. Al final, la interpretación del agente sobre el mensaje del

usuario y sobre las herramientas es donde reside la verdadera complejidad y el alma de este TFM, ya que a partir de la generación de una hoja de ruta y flujo de trabajo correctos el resto reside en el propio código cuyos posibles errores son más sencillos de solucionar. A partir de cuantiosas pruebas se ha conseguido llegar a formar una arquitectura que funciona como un bloque unido cuya sintonía se muestra en la demostración.

El quinto y último de los objetivos específicos, propone examinar a través de los mapas resultantes y las métricas de rendimiento y consumo al sistema las posibles limitaciones y líneas de mejora. Después de las pruebas realizadas se verificó la viabilidad del sistema al tener unos tiempos de ejecución y consumo de tokens más que razonables para diferentes casuísticas según la densidad de puntos y si la ejecución se lanzaba con la aplicación ya iniciada o no. Por otra parte, los mapas mostraron patrones estables para las diferentes configuraciones con una mayor continuidad en la interpolación espacial en aquellos donde la densidad de puntos es mayor, lo que también supone como así reflejan los resultados, un aumento en el tiempo y tokens consumidos, aunque siempre dentro de un margen razonable. En cuanto a las limitaciones y las líneas de mejora se detallarán de manera más explícita en los subapartados posteriores de Limitaciones y Prospectiva.

Finalmente, el objetivo general establecía desarrollar un sistema con un agente orquestador basado en inteligencia artificial generativa que demostrara su viabilidad y eficiencia operativa como herramienta funcional para el análisis de riesgo de incendios. A la luz de la calidad del sistema creado y de sus resultados, el objetivo general se considera cumplido en su totalidad. El sistema implementado integra razonamiento autónomo y ejecución de herramientas para producir, en tiempo real, indicadores y visualizaciones geoespaciales relevantes en el ámbito de prevención de incendios. A su vez, muestra una relación comprensible entre entradas, procesos y salidas y opera con tiempos y costes confirma la viabilidad técnica y la utilidad potencial del proyecto.

En conjunto, el cumplimiento paso a paso de cada uno de los objetivos específicos cumplidos conduce de manera directa al logro del propósito global,

dejando establecida una base sólida que demuestra su potencial para una implementación futura ya con un carácter profesional asociado.

1. Limitaciones

A pesar de los resultados satisfactorios obtenidos y la viabilidad demostrada, el proyecto también presenta una serie de limitaciones que deben ser reconocidas. Es cierto que estas limitaciones no invalidan los logros alcanzados, sin embargo, se considera de gran importancia señalar los aspectos que deben ser revisados, solucionados o mejorados en futuros desarrollos para poder evolucionar hacia una herramienta operativa en entornos profesionales. A continuación, se exponen los puntos principales de cara a este aspecto.

En primer lugar, una de las limitaciones más relevantes y que se ha comentado explícitamente en apartados anteriores se refiere a la simplificación de los cálculos físicos y ambientales empleados en el modelo en la herramienta de cálculo de la probabilidad de ignición. El primer motivo de esta limitación es debido a la dificultad de obtener ciertos datos como la variabilidad en la humedad del combustible fino muerto, la inclinación del terreno o los factores de corrección por exposición solar y orientación del viento. El segundo y último motivo no es otro que la complejidad que esto conllevaba y dado el tiempo que había disponible para la realización de las prácticas en la empresa XRF, se prefirió centrarse en el objetivo final del proyecto usando unos cálculos aproximados. La ausencia de estas variables en el cálculo del riesgo limita la capacidad del agente para ofrecer predicciones con resultados exactos, no obstante, se aseguró de que fueran unas buenas aproximaciones dado que se consideró el consejo de un profesional en este ámbito.

En segundo lugar, el sistema presenta limitaciones asociadas a la dependencia de fuentes de datos externas, concretamente APIs meteorológicas abiertas como Open-Meteo y servicios de geocodificación como Nominatim. Aunque estas herramientas destacan por su accesibilidad y

carácter gratuito, al final son herramientas que dependen de terceros y no son específicas de nuestro caso de uso. En caso de querer llevar este proyecto a un ámbito profesional requeriría tener acceso a las mismas bases de datos a las que acceden los profesionales de emergencias para así partir de los mismos datos y poder garantizar unos resultados lo más exactos posibles y sobre todo con las mismas fuentes de información. Además, esta limitación también afecta de manera derivada a la limitación expuesta anteriormente. Por tanto, la dependencia de bases de datos externas y no especializadas en el marco de prevención de incendios supone que los resultados no estén del todo alineados con la realidad.

En tercer lugar, se identifican limitaciones de tipo computacional y de rendimiento, derivadas tanto de la infraestructura disponible como del modelo de lenguaje utilizado. Como se ha comentado durante el trabajo, se ha utilizado el modelo GPT-4-omini dentro del servicio Azure OpenAI, un opción eficiente y accesible para un estudiante. Sin embargo, en un contexto donde exista un presupuesto dedicado, sería posible emplear modelos más optimizados, con capacidad de mejorar la habilidad del agente, reducir los tiempos de razonamiento y generar ejecuciones con una mayor precisión. En definitiva, el rendimiento observado refleja un equilibrio razonable entre coste, precisión y capacidad, pero también evidencia un margen de mejora directamente vinculado a los recursos computacionales disponibles.

Por último, se debe señalar una limitación funcional en la app de Streamlit de un mecanismo integrado de diálogo posterior a la generación de mapas. Además de la generación de mapas, el agente es capaz de responder a cualquier pregunta que esté a su alcance a través de las herramientas implementadas, es por eso por lo que se echa en falta un espacio donde el usuario pueda interactuar con un input personalizado en lenguaje natural y solicitar respuestas sobre datos o preguntas relacionadas, sería como un chat al margen que respondiera dudas e inquietudes concretas. En resumen, la integración de un módulo conversacional dentro de la interfaz permitiría ampliar considerablemente la utilidad del prototipo, creando un entorno en el que el usuario pudiera alternar entre consultas visuales y textuales.

En conjunto, estas limitaciones constituyen un paso necesario para consolidar el sistema hacia niveles superiores de precisión, estabilidad y funcionalidad y es el primer paso para la posible implementación de futuras líneas de mejora que se muestra a continuación.

2. Futuras líneas de mejora

Las acciones de mejora propuestas guardan una estrecha relación con el apartado anterior de limitaciones ya que derivan de las mismas. Entre ellas destacan la incorporación de modelos más completos para el cálculo de ignición y peligro, integrando variables actualmente simplificadas como la pendiente del terreno o la carga de combustible. También se considera sustituir o complementar las fuentes de datos abiertas por sistemas meteorológicos de mayor precisión orientados al caso de uso en concreto. En el ámbito computacional, una futura versión del sistema debería trabajar con modelos de lenguaje de mayor capacidad, capaces de disminuir la latencia del razonamiento y gestionar mallas más densas sin comprometer el rendimiento. Igualmente, la interfaz podría ampliarse incorporando un módulo conversacional tras la generación de mapas, de modo que el usuario pudiera solicitar respuestas adicionales. Además, para que se pueda extraer todo el potencial a esta última mejora sería necesario dotar al agente de una memoria para que pueda responder teniendo en cuenta información de consultas anteriores o de los mapas recientemente generados.

Más allá de estas optimizaciones funcionales, se presenta una evolución más prometedora del proyecto que consiste en una reestructuración de la arquitectura hacia un modelo distribuido basado en una red de agentes especializados. En lugar de un único agente que centraliza todas las funciones, se podría desarrollar un sistema en el que cada agente desempeñe un rol concreto con un conjunto definido de herramientas. Un primer grupo podría estar compuesto por agentes de búsqueda y obtención de datos, dotados de diversas APIs meteorológicas, geoespaciales y bases de datos locales. Un segundo grupo incluiría agentes de cálculo, especializados en ejecutar modelos de cálculo avanzados y un tercer agente destinado a la visualización

que actuaría como responsable de generar mapas y gráficos integrando los resultados previamente procesados. Sobre esta red actuaría un agente orquestador general, encargado de coordinar la comunicación entre estos tres agentes y distribuir tareas comprobando la coherencia entre resultados. Este enfoque favorecería la escalabilidad y sobre todo permitiría añadir nuevas capacidades al sistema haciéndolo mucho más complejo y teniendo la habilidad de abordar tareas de mayor dificultad. Además, abriría la puerta a incorporar técnicas de coordinación multiagente más sofisticadas, capaces de adaptar dinámicamente los recursos utilizados como pueden ser el protocolo MCP o un uso más presente de la arquitectura de nodos disponible a partir de LangGraph.

En conjunto, este modelo distribuido representa una evolución natural hacia sistemas más complejos y aptos para integrarse en entornos profesionales de distinta naturaleza.

3. Consideraciones finales

El desarrollo de un sistema como el agente de emergencias demuestra el gran potencial de la inteligencia artificial generativa y su capacidad para adaptarse a multitud de casos de uso como es en este caso la predicción de riesgos en entornos de emergencia. Personalmente, me siento afortunado de haber trabajado con un tema tan novedoso y con una versatilidad y polivalencia tan alta, lo cual no solo ha ampliado mi perspectiva sobre la inteligencia artificial, sino que me ha permitido trabajar en un campo con un impacto directo en la seguridad pública y la protección del medio ambiente.

En cuanto a las competencias adquiridas, este proyecto ha sido una experiencia enriquecedora tanto técnicamente como personalmente. He logrado profundizar en el mundo de la inteligencia artificial generativa, aprendiendo a integrar agentes autónomos con múltiples herramientas, como APIs en tiempo real, y desarrollar una solución eficiente. A nivel metodológico, el proceso de diseño, implementación y evaluación me ha permitido mejorar mis habilidades en programación, análisis de datos y sobre todo desarrollo de sistemas multiagentes complejos. Asimismo, el trabajo en

equipo con los profesionales de XRF.ai ha sido indispensable, permitiéndome aplicar mis conocimientos en un entorno real de trabajo y guiándome a medida que iban apareciendo nuevos retos y obstáculos a lo largo del trabajo.

El hecho de ser ingeniero de Telecomunicaciones hace que siempre estuviera interesado en la tecnología, pero fue al investigar por mi cuenta sobre inteligencia artificial cuando surgió en mí un gran interés y curiosidad. El Máster en Inteligencia Artificial ha sido clave para enfocar mi carrera hacia un sector que considero fundamental para el futuro de la tecnología. A lo largo del máster, no solo he ampliado mis conocimientos técnicos, sino que también he reforzado mi convicción de que la IA es el camino que quiero seguir profesionalmente, especialmente dentro del campo de la IA Generativa.

En cuanto a mi autoevaluación, siento que he mejorado en la gestión de proyectos y en la resolución de problemas complejos, aunque considero que el gran paso llegará cuando pueda realizar un proyecto centrado en un entorno profesional y real. Este máster ha consolidado mi interés por la IA pero sobre todo ha sentado las bases teóricas de todo este mundo, impulsándome a buscar una oportunidad en este sector a partir de la cual poder retarme personalmente y a su vez contribuir al desarrollo de este fascinante campo.

Quisiera cerrar este trabajo expresando mi sincero agradecimiento. En primer lugar, a los profesores de la UAX, cuyo compromiso y conocimientos han sido fundamentales en mi formación. También deseo agradecer al jefe del departamento de IA de XRF.ai, que actuó como tutor de prácticas y me acompañó durante todo el proceso con orientación constante.

En un plano más personal, quiero dar las gracias a mis amigos y familiares, y en especial a mis padres y a Celia, mi pareja, por su apoyo diario e incondicional. Sin la ayuda y la confianza de todas estas personas, este proyecto no habría podido llevarse a cabo.

Bibliografía

- XRF - AI and XR for smarter real-time decision making. (n.d.). <https://www.xrf.ai/>
- Agents - Docs by LangChain. (n.d.). Docs by LangChain. <https://docs.langchain.com/oss/python/langchain/agents>
- Aahill. (n.d.). How to use connected agents - Microsoft Foundry. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/ai-foundry/agents/how-to/connected-agents?view=foundry-classic&pivots=portal>
- Deepchecks. How Multi-Agent LLMs Differ from Traditional LLMs. <https://www.deepchecks.com/how-multi-agent-llms-differ-from-traditional-llms/>
- AI Agents vs. Multi-agent systems: From solo expertise to orchestrated collective intelligence | Talan - Site groupe. (n.d.). TALAN. <https://www.talan.com/global/en/ai-agents-vs-multi-agent-systems-solo-expertise-orchestrated-collective-intelligence/>
- A survey on Multi-Generative Agent System: Recent advances and new frontiers. (n.d.). <https://arxiv.org/html/2412.17481v1>
- Gutowska, A., & Stryker, C. (2025, November 19). AI Agents. IBM. <https://www.ibm.com/think/ai-agents#605511093>
- LangChain. (n.d.). Agents | LangChain Reference. https://python.langchain.com/docs/modules/agents/agent_types/react/

- OpenAI. (2024). Agents: Designing systems that can use tools and take actions. <https://platform.openai.com/docs/guides/agents>
- European Forest Fire Information System (EFFIS). Forest Fire Risk and Danger Indexes in Europe. <https://effis.jrc.ec.europa.eu>
- Incendios forestales. (n.d.). Ministerio Para La Transición Ecológica Y El Reto Demográfico. <https://www.miteco.gob.es/es/biodiversidad/temas/incendios-forestales/>
- Free Open-Source Weather API | Open-Meteo.com. (n.d.). <https://open-meteo.com/>
- Steidani. (n.d.). GitHub - steidani/FireDanger: Forest Fire Danger - Calculation of indices for forest fire risk assessment in weather and climate data. GitHub. <https://github.com/steidani/FireDanger>
- Hugging Face. (2024). *Tools – Smolagents Documentation (v0.1.3)*. Hugging Face Docs. <https://huggingface.co/docs/smolagents/v0.1.3/tutorials/tools>
- Hugging Face. (2024, enero 30). *Introducing Smolagents: Simple Agents That Write Actions in Code*. Hugging Face Blog. <https://huggingface.co/blog/smolagents>
- LangChain AI. (2024). *How to Create a ReAct Agent from Scratch (Functional API)*. LangGraph Documentation. <https://langchain-ai.github.io/langgraph/how-tos/ReAct-agent-from-scratch-functional/>

- Mocnik, F.-B., Zipf, A., & Raifer, M. (2017). *The OpenStreetMap data model and its application in geocoding and routing*. ISPRS International Journal of Geo-Information, 6(2), 54. <https://doi.org/10.3390/ijgi6020054>
- Stål, T., & Reading, A. M. (2020). *A Grid for Multidimensional and Multivariate Spatial Representation and Data Processing*. *Journal of Open Research Software*, 8(2). <https://doi.org/10.5334/jors.287>
- Jaquenoud, T. (2021). *AI Geospatial Wildfire Risk Prediction*. Kaggle. <https://www.kaggle.com/code/thjaquenoud/ai-geospatial-wildfire-risk-prediction>
- Talaviya, A. (2022, diciembre 28). *A Complete Guide to Building Streamlit Data Web Apps with Code Examples*. Medium. <https://medium.com/@avikumart /a-complete-guide-to-building-streamlit-data-web-apps-with-code-examples-6cc1e42b2397>



Anexos

Tablas de probabilidad de ignición y peligro de incendio utilizadas por las unidades de emergencia de La Palma

TABLA II.1

HUMEDAD BÁSICA DEL COMBUSTIBLE FINO MUERTO
Día: De 8:00 a 20:00 horas (solar)
HUMEDAD RELATIVA DEL AIRE (porcentaje)

Temp. Termómetro Seco (°C)	0	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100
<0	4	9	14	19	24	29	34	39	44	49	54	59	64	69	74	79	84	89	94	99	100
0-9	1	2	2	3	4	5	5	6	7	8	8	9	9	10	10	11	12	12	13	13	14
10-20	1	2	2	3	4	5	5	6	7	7	8	8	9	9	10	10	11	12	13	13	13
21-31	1	1	2	2	3	4	5	5	6	7	7	8	8	9	9	10	11	12	12	13	13
32-42	1	1	2	2	3	4	4	5	6	7	7	8	8	9	10	10	11	12	12	13	13
>42	1	1	2	2	3	4	4	5	6	7	7	8	8	9	10	10	11	12	12	13	13

Añadir el sumando corrector que corresponda según las Tablas II.2, II.3 o II.4.

TABLA II.2

SUMANDO CORRECTOR DEL CONTENIDO DE HUMEDAD DEL COMBUSTIBLE
(Mayo-Junio-Julio)
Día: De 08:00 a 20:00 horas (solar)

Expuesto. Menos del 50% de los combustibles en sombra

Exposición	Pendiente	08:00	10:00	12:00	14:00	16:00	18:00	20:00
N	0-30%	3	1	0	0	1	3	
	>30%	4	2	1	1	2	4	
E	0-30%	2	1	0	0	1	4	
	>30%	2	0	0	1	3	5	
S	0-30%	3	1	0	0	1	3	
	>30%	3	1	1	1	1	3	
O	0-30%	3	1	0	0	1	3	
	>30%	5	3	1	0	0	2	
Sombreado. Más del 50% de los combustibles en sombra o nublado								
N	0% +	5	4	3	3	4	5	
E	0% +	4	4	3	4	4	5	
S	0% +	4	4	3	3	4	5	
O	0% +	5	4	3	3	4	4	

Terreno llano = Exposición Sur.

TABLA II.3

SUMANDO CORRECTOR DEL CONTENIDO DE HUMEDAD DEL COMBUSTIBLE
(Febrero-Marzo-Abril-Agosto-Septiembre-October)
Día: De 08:00 a 20:00 horas (solar)

Expuesto. Menos del 50% de los combustibles en sombra.

Exposición	Pendiente	08:00	10:00	12:00	14:00	16:00	18:00	20:00
N	0-30%	4	2	1	1	2	4	
	>30%	4	3	3	3	3	4	
E	0-30%	4	2	1	1	2	4	
	>30%	3	1	1	2	4	5	
S	0-30%	4	2	1	1	2	4	
	>30%	4	2	1	1	2	4	
O	0-30%	4	2	1	1	2	4	
	>30%	5	4	2	1	1	3	
Sombreado. Más del 50% de los combustibles en sombra o nublado								
N	0% +	5	5	4	4	5	5	
E	0% +	5	4	4	4	5	5	
S	0% +	5	4	4	4	4	5	
O	0% +	5	5	4	4	4	5	

Terreno llano = Exposición Sur.

TABLA II.4

**SUMANDO CORRECTOR DEL CONTENIDO DE HUMEDAD DEL COMBUSTIBLE
(Noviembre-Diciembre-Enero)**
Día: De 08:00 a 20:00 horas (solar)

Expuesto. Menos del 50% de los combustibles en sombra.

Exposición	Pendiente	08:00	10:00	12:00	14:00	16:00	18:00	20:00
N	0-30%	5	4	3	3	4	5	
	>30%	5	5	5	5	5	5	
E	0-30%	5	4	3	3	4	5	
	>30%	5	4	3	2	5	5	
S	0-30%	5	4	3	2	4	5	
	>30%	5	3	1	1	3	5	
O	0-30%	5	4	3	3	4	5	
	>30%	5	5	4	2	3	5	
Sombreado. Más del 50% de los combustibles en sombra o nublado.								
Todas las exposiciones y pendientes		5	5	5	5	5	5	
Terreno llano = Exposición Sur.								

TABLA II.5

**HUMEDAD DEL COMBUSTIBLE
Noche: De 20:00 a 08:00 horas (solar)**
HUMEDAD RELATIVA DEL AIRE (porcentaje)

Temp. Termómetro Seco (°C)	0	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100
0-9	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
10-20	4	9	14	19	24	29	34	39	44	49	54	59	64	69	74	79	84	89	94	99	100
21-31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
32-42	1	2	3	3	4	5	6	7	8	9	10	10	11	12	13	14	15	16	17	18	19
>42	1	2	3	3	4	5	6	6	9	9	9	9	10	11	12	14	16	19	21	24	25

No precisa corrección.

Estimación de la velocidad del viento sin anemómetro
ESCALA BEAUFORT PARA LA FUERZA DEL VIENTO

Grado Beaufort	Término descriptivo	Efectos en tierra	Velocidad En Km/h.
0	Calma	El humo sube verticalmente.	<2
1	Ventolina	La veleta no se mueve apenas. El viento inclina ligeramente el humo que asciende.	2-5
2	Flojito	Perceptible en la cara. Agita hojas. La veleta se mueve.	5-10
3	Flojo	Hojas y ramillas se mueven sin cesar. Ondeán los gallardetes.	10-20
4	Bonancible o moderado	Levanta papeles y polvo. Agita ramitas desnudas.	20-30
5	Fresquito	Agita pequeños árboles con hojas y ramas medianas desnudas. En lagos y estanques, pequeñas ondas con cresta.	30-40
6	Fresco	Mueve ramas gruesas. Silban los alambres del telégrafo. Es difícil usar el paraguas.	40-50
7	Frescachón	Agita árboles medianos. Es difícil trabajar o moverse bajo el viento.	50-60
8	Temporal	Mueve árboles gruesos y arranca pequeñas ramas. Es imposible caminar bajo el viento.	60-75
9	Temporal fuerte	Desgaja ramas medianas, arranca tejós y chimeneas, derriba cornisas y macetas.	75-85
10	Temporal duro	Poco frecuente en tierra. Troncha y arranca árboles. Graves daños en los edificios.	85-100

TABLA PSICROMETRICA: Humedad relativa del aire (%)
Diferencia entre termómetro seco y termómetro húmedo (°C)

Temperature	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22					
0	100	90	80	72	64	56	48	42	35	30	24	19	15	10	5													
1	100	90	80	73	64	55	48	42	35	30	24	19	15	10	5													
2	100	90	81	73	64	55	48	42	35	30	24	19	15	10	5													
3	100	91	83	74	65	57	48	42	35	30	24	19	15	10	5													
4	100	91	83	74	66	59	50	43	35	30	24	19	15	10	5													
5	100	92	84	76	66	59	52	48	38	31	24	19	15	10	5													
6	100	92	85	77	70	62	55	50	41	34	27	20	15	10	5													
7	100	92	85	77	71	63	56	52	44	37	30	24	19	12	5													
8	100	93	86	78	72	63	58	53	46	40	33	27	21	16	9	5												
9	100	93	86	78	72	63	58	53	46	40	33	27	21	16	9	5												
10	100	93	86	78	72	63	58	53	46	40	33	27	21	16	10	5												
11	100	93	87	80	74	68	62	56	50	44	38	33	27	21	16	10	5											
12	100	93	87	82	76	69	62	57	52	46	40	35	30	24	19	13	8	4										
13	100	93	88	82	77	71	65	60	55	50	45	40	35	30	25	20	15	10	6									
14	100	94	88	83	77	72	66	61	56	52	47	42	37	32	27	22	18	13	9	6								
15	100	94	88	83	76	73	67	62	57	53	48	44	39	34	30	25	21	17	12	8	5							
16	100	94	89	83	78	74	68	63	58	53	48	44	39	34	30	25	21	17	12	8	5							
17	100	94	89	84	80	75	70	64	60	56	52	48	43	38	34	30	24	22	18	14	10	7						
18	100	95	90	85	80	76	71	66	61	57	53	49	45	40	37	32	28	24	20	16	15	10	6					
19	100	95	90	86	81	77	72	68	62	58	55	51	47	42	38	34	30	27	23	20	16	15	12	8				
20	100	95	90	86	82	77	72	68	63	60	56	52	48	44	41	37	32	29	25	22	18	15	11	8				
21	100	95	91	86	82	78	73	69	64	60	57	53	49	46	42	38	34	31	27	24	21	17	15	11	7			
22	100	95	91	87	83	78	74	70	66	62	58	54	51	47	43	40	37	33	29	26	23	20	18	13	10	7		
23	100	95	91	87	83	79	75	70	66	63	58	56	52	48	46	41	38	35	31	28	25	22	19	17	15	12	10	7
24	100																											

ÍNDICE DE PELIGRO

Se determina calculando:

La probabilidad de ignición (Tabla III), utilizando la humedad del combustible fino muerto estimado en las Tablas I y II.

El estado de alerta (Tabla IV).

TABLA III

PROBABILIDAD DE IGNICIÓN

(Estima la probabilidad de que una pavesa o brasa, al caer sobre el combustible ligero muerto, pueda inflamarlo)

Sombreado (%)		Temperatura Termómetro Seco, °C	HUMEDAD DEL COMBUSTIBLE LIGERO MUERTO (%)															
			2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0-10	40+	100	100	90	80	70	60	50	40	40	30	30	20	20	20	10	10	
	35-40	100	90	80	70	60	60	50	40	40	30	30	20	20	20	10	10	
	30-35	100	90	80	70	60	50	50	40	30	30	30	20	20	20	10	10	
	25-30	100	90	80	70	60	50	40	40	30	30	20	20	20	20	10	10	
	20-25	100	80	70	60	60	50	40	40	30	30	20	20	20	10	10	10	
	15-20	90	80	70	60	50	50	40	30	30	30	20	20	20	10	10	10	
	10-15	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	5-10	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	0-5	90	70	60	60	50	40	40	30	30	20	20	20	10	10	10	10	
10-50	40+	100	100	80	70	60	60	50	40	40	30	30	20	20	20	10	10	
	35-40	100	90	80	70	60	50	50	40	40	30	30	20	20	20	10	10	
	30-35	100	90	80	70	60	50	40	40	30	30	30	20	20	20	10	10	
	25-30	100	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	
	20-25	100	80	70	60	50	50	40	40	30	30	20	20	20	10	10	10	
	15-20	90	80	70	60	50	50	40	30	30	20	20	20	20	10	10	10	
	10-15	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	5-10	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	0-5	80	70	60	50	50	40	30	30	20	20	20	10	10	10	10	10	
50-90	40+	100	90	80	70	60	50	50	40	40	30	30	20	20	20	10	10	
	35-40	100	90	80	70	60	50	50	40	30	30	30	20	20	20	10	10	
	30-35	100	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	
	25-30	100	80	70	60	60	50	40	40	30	20	20	20	20	10	10	10	
	20-25	90	80	70	60	50	50	40	30	30	30	20	20	20	10	10	10	
	15-20	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	10-15	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	5-10	90	70	60	50	50	40	30	30	30	20	20	20	10	10	10	10	
	0-5	80	70	60	50	50	40	30	30	20	20	20	10	10	10	10	10	
90-100	40+	100	90	80	70	60	50	50	40	30	30	30	20	20	20	10	10	
	35-40	100	90	80	70	60	50	40	40	30	30	20	20	20	20	10	10	
	30-35	100	80	70	60	60	50	40	40	30	30	20	20	20	10	10	10	
	25-30	90	80	70	60	50	50	40	30	30	30	20	20	20	10	10	10	
	20-25	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	15-20	90	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	
	10-15	90	70	60	60	50	40	40	30	30	20	20	20	10	10	10	10	
	5-10	80	70	60	50	50	40	30	30	20	20	20	10	10	10	10	10	
	0-5	80	70	60	50	40	40	30	30	20	20	20	10	10	10	10	10	

TABLA IV
INTERPRETACIÓN DEL ÍNDICE DEL PELIGRO

Zonas del interior y áreas costeras		Vientos no terrales			
Probabilidad de ignición (%)		Velocidad del viento (km/h)			
	0-9	10-19	20-39	≥ 40	
10≤20	Prealerta	Prealerta	Prealerta	Alerta	
20≤50	Prealerta	Alerta	Alerta	Alerta	
50<70	Alarma	Alarma	Alarma	Alarma	
≥70	Alarma	Alarma	Alarma	Alarma extrema	
Áreas costeras		Vientos terrales			
Probabilidad de ignición (%)		Velocidad del viento (km/h)			
	0-9	10-19	20-39	≥ 40	
10≤20	Prealerta	Alerta	Alerta	Alerta	
20≤50	Alerta	Alarma	Alarma	extrema	
50<70	Alarma	Alarma	Alarma	Alarma	
≥70	Alarma	Alarma extrema	Alarma extrema	Alarma extrema	
Prealerta:	Peligro bajo o moderado. Sin precauciones especiales.				
Alerta:	Peligro moderado. Los medios estarán listos para ser movilizados.				
Alarma:	Peligro alto. La vigilancia preventiva será intensificada. El paso a las zonas boscosas podrá ser limitado. Los medios de lucha estarán preparados al máximo. Se informará a la población a través de los medios de comunicación para que adopte medidas preventivas.				
Alarma extrema:	Peligro extremo. Altísima probabilidad de múltiples y grandes incendios. Formación de focos secundarios causados por pavesas. No debe ser permitido ningún punto de fuego en las cercanías del monte (hogueras, parrillas, quemados agrícolas, basureros, etc.). Se limitará al máximo el paso al monte. Las pistas forestales se cortarán. Todos los medios estarán preparados al máximo. Se informará a la población a través de los medios de comunicación para que adopte medidas preventivas.				

ÍNDICE DE PELIGRO:

VALOR	CÓDIGO
Prealerta	1
Alerta	2
Alarma	3
Alarma Extrema	4

En caso de incendios lejanos a la base, conviene utilizar como valor más representativo de la probabilidad de ignición e índice de peligro, el obtenido por la base de medios aéreos más próxima al incendio, o en su defecto, el suministrado por la Comunidad Autónoma, a través de sus Servicios provinciales o de su Central de Operaciones. Para misiones de incendio **NUNCA** hay que dejar vacío este campo.